题型

题型：单项选择，基本概念(回答限制在30字以内)，综合题（包括编程和计算）

并发

线程：

共享内存的执行流；

拥有独立的“上下文”和栈帧列表；

共享全局变量、堆空间。

安全性

No Bad things happen

活性

Good things finally come true.

需要的条件：

互斥(safety)、行进(liveness)、有界等待(no starvation)

互斥

什么是互斥?

临界区 最多只有一个线程

如何实现?

使用 锁 实现

(一) 基于硬件的:

自旋锁 & 叫号锁

(二) 基于 系统调用 的锁

直接用可能会唤醒丢失

(三) 使用futex解决唤醒丢失

futex_wait(addr,v); if addr == v: 睡;

否则futex_wait函数直接返回



补充:

一些原子指令:

fetch_add(p,value)

ov = *p

(*p += value)

return ov

cmp_swap(p,oldV,newV)

if p == oldV: p = newV

return oldV

test_set(p,v)

ov = p

*p = v

return ov

条件变量

消费者 生产者 问题

```
int cnd1 = 0;  //生产者能够生产的条件
int cnd2 = 0;  //消费者能够生产的条件


void produce() {
    while(1) {
    retry:
        mutex_lock(&lk);
        int ready = (depth < n);
        if (!ready){
                int val = cnd1;
                mutex_unlock(&lk);
                futex_wait(&cnd1, val);
                goto retry;
        }

        printf("(");
        depth++;
        atomic_add(&cnd2, 1);
        futex_wake(&cnd2);
        mutex_unlock(&lk);
    }
```

```
void consume(){
    while(1){
    retry:
        mutex_lock(&lk);
        int ready = (depth > 0);
        if (!ready) {
             int val = cnd2;
             mutex_unlock(&lk);
             futex_wait(&cnd2, val);
             goto retry;
        }

        printf(")");
        depth--;
        atomic_add(&cnd1, 1);
        futex_wake(&cnd1);
        mutex_unlock(&lk);
    }
}
```

我们实现了条件变量(Condition Variable)

- 条件变量：用于标记某个用于同步条件的变量， 其有两个相关的操作：
  - cond_wait(cond_t *cv, mutex_t *lk)
    - 调用之前默认假设lk已经持有锁lk
    - 调用之后原子的阻塞线程和释放锁
    - 被唤醒时重新acquire the lk
  - cond_signal(cond_t *cv)
    - 唤醒一个等在条件变量cv上的一个阻塞线程
    - 如果没有线程阻塞在这个条件变量上，什么也不做

```
typedef struct condtional_varaible {
    unsigned value;
}cond_t;

void cond_wait(cond_t *cv, mutex_t *lk)
{
        int val = atomic_load(&cv->value);

        mutex_unlock(lk);
        futex_wait(&cv->value, val);
        mutex_lock(lk);
}
```

```
void cond_signal(cond_t *cv)
{
        atomic_fetch_add(&cv->value, 1);
        futex_wake(&cv->value);
}
```

*注：这里存在一个微妙的可能错误（包括之前的解决方案4）：整数会溢出，因此实际实现更加复杂

C-P 问题 By Condtion-Variable



```
cond_t cv_p = COND_INIT();
cond_t cv_c = COND_INIT();
mutex_t lk = MUTEX_INIT();

void produce() {
    while(1) {
        mutex_lock(&lk);
        if(!(depth < n)){
            cond_wait(&cv_p, &lk);
        }

        printf("(");
        depth++;
        cond_signal(&cv_c);
        mutex_unlock(&lk);
    }
}
```

```
void consume() {
    while(1) {
        mutex_lock(&lk);
        if(!(depth > 0)){
            cond_wait(&cv_c, &lk);
        }

        printf(")");
        depth--;
        cond_signal(&cv_p);
        mutex_unlock(&lk);
    }
}
```

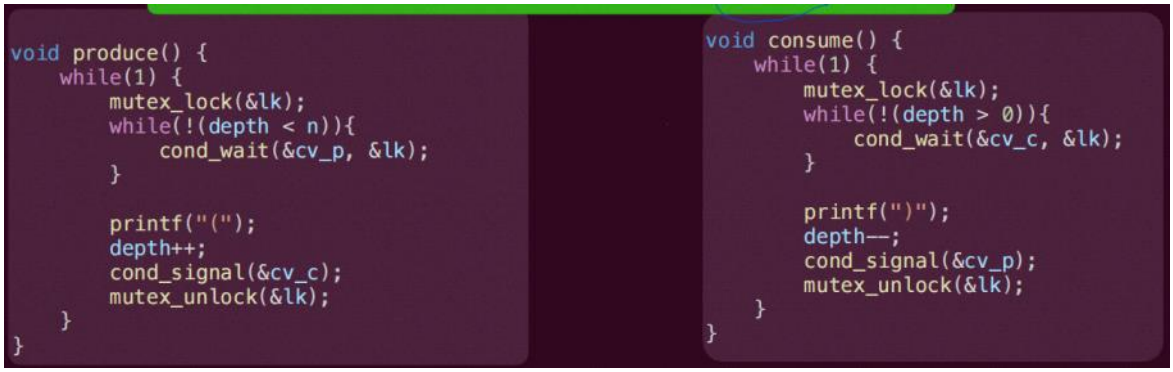
cond_signal和cond_wait之间在实践中是非原子语义，

即：唤醒和锁的转移不是原子的！也就是说，被唤醒的线程在获取锁之后，不能保证被唤醒的条件依旧成立。这，会导致错误：

考虑如下情况：一个生产者p，两个消费者 c1,c2,

首先消费者c1等待一个空的buffer，然后生产者p生产一个数据，调用唤醒函数唤醒c1，但此时如果c2抢先一步进入临界区，并消费掉了数据后返回，然后c1才被真正唤醒进入临界区，但此时，临界区已经没有数据可以消费了，错误!

为之奈何？=> 唤醒后需要检查，改为while。



```
void produce() {
    while(1) {
        mutex_lock(&lk);
        while(!(depth < n)){
            cond_wait(&cv_p, &lk);
        }

        printf("(");
        depth++;
        cond_signal(&cv_c);
        mutex_unlock(&lk);
    }
}
```

```
void consume() {
    while(1) {
        mutex_lock(&lk);
        while(!(depth > 0)){
            cond_wait(&cv_c, &lk);
        }

        printf(")");
        depth--;
        cond_signal(&cv_p);
        mutex_unlock(&lk);
    }
}
```

大力出奇迹——cond_broadcast 唤醒所有线程

信号量

条件变量 需要程序员手动进行额外的条件判定，

“麻烦！不够简单！”

一个自带“整数”的同步语句就是信号量。

P(sem_t * sem)

--- Decrease if positive, else yield

V(sem_t * sem)

--- Increase, if waiters > 0, wake up one thread

（一）使用 互斥锁+条件变量 实现信号量



```
typedef struct  _sem_t {
    int value;
    pthread_cond_t cond;
    pthread_mutex_t lock;
} sem_t;

// only one thread can call this
void sem_init(sem_t *s, int value) {
    s->value = value;
    cond_init(&s->cond);
    mutex_init(&s->lock);
}
```

```
void sem_wait(sem_t *s) {
    mutex_lock(&s->lock);
    while (s->value <= 0)
        cond_wait(&s->cond, &s->lock);

    s->value--;
    mutex_unlock(&s->lock);
}
```

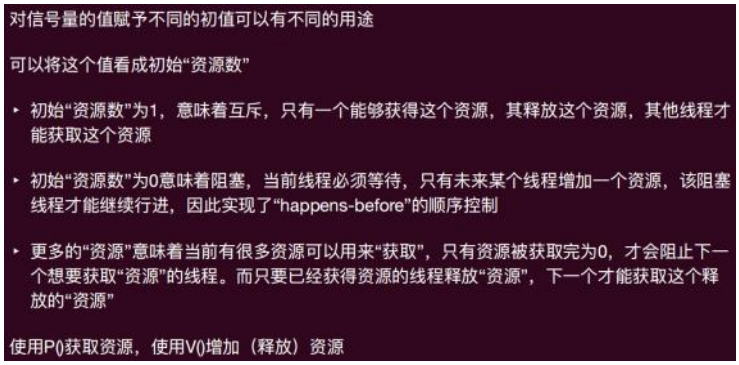
```
void sem_post(sem_t *s) {
    mutex_lock(&s->lock);
    s->value++;
    cond_signal(&s->cond);
    mutex_unlock(&s->lock);
}
```

（二）posix中

int sem_init(sem_t *sem, int pshared, unsigned int value);

- sem：指向 sem_t 类型的指针，用于存储新创建的信号量。
- pshared：一个整数，指示信号量是否在多个进程间共享。如果设置为 0，则信号量仅在调用进程的线程之间共享；如果设置为 PTHREAD_PROCESS_SHARED，则信号量可以在具有相同信号量对象的进程之间共享。
- value：信号量的初始值，通常设置为 0 或 1。0 表示没有资源可用，1 表示有一个资源可用。

（三）信号量的初始值



（四）锁 存于 信号量中

特别注意：PV完成后的代码是无锁保护的，访问临界区依旧要加锁。


sem_t mutex; //用于互斥的信号量（也可以用互斥锁）
sem_init(sem_t &mutex, 0, 1);
sem_t empty; //缓冲区有多少空
sem_init(sem_t &empty, 0, MAX);
sem_t full; //缓冲区有多少资源
sem_init(sem_t &full, 0, 0);
void produce() {
while(1) {
P(&mutex);
P(&empty);
produce_shared_buffer();
V(&full);
V(&mutex);
}
}
产生死锁：如果消费者先进行互斥，然后消费，发现没有数据，阻塞，但生产者由于互斥锁此时无法获得，无法进入临界区产生数据，因此生产者和消费者都阻塞，从而无法行进
void consume() {
while(1) {
P(&mutex);
P(&full);
consume_shared_buffer();
V(&empty);
V(&mutex);
}
}
sem_t mutex; //用于互斥的信号量（也可以用互斥锁）
sem_init(sem_t &mutex, 0, 1);
sem_t empty; //缓冲区有多少空
sem_init(sem_t &empty, 0, MAX);
sem_t full; //缓冲区有多少资源
sem_init(sem_t &full, 0, 0);
void produce() {
while(1) {
P(&empty);
P(&mutex);
produce_shared_buffer();
V(&mutex);
V(&full);
}
}
条件允许，才进入临界区
void consume() {
while(1) {
P(&full);
P(&mutex);
consume_shared_buffer();
V(&mutex);
V(&empty);
}
}

死锁

小组中的成员因为每个成员都在等待另一个成员(include itself)采取行动

（一）A-A型 死锁

已经有锁，仍然尝试获取

方法论：

xv6防御性编程，check：if (holding(lk)) {panic("reacquire!")}

（二）ABBA 死锁

T1 拿到A 要B；T2 拿到B 要A



（三）死锁的必要条件

资源互斥或有限

持有某个资源并等待更多资源

不能直接抢别人的资源，只能等他人主动释放

形成循环等待的环

（四）处理方式

忽略，重启大法

源头避免

检测恢复

（五）死锁避免——破坏必要条件

破坏互斥：

问题：不容易，无锁算法需要很小心

破坏持有等待：

i. 所有或者一无所有（）

ii. 没有所有不如一无所有（）

破坏非抢占性：

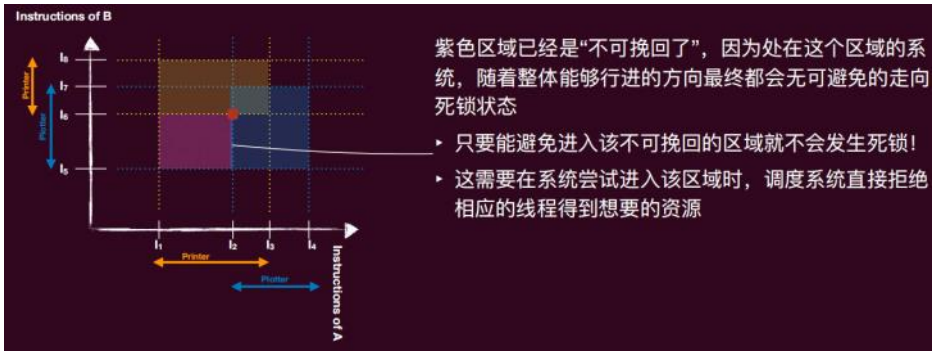
问题：不是所有都能抢

破坏环形等待：

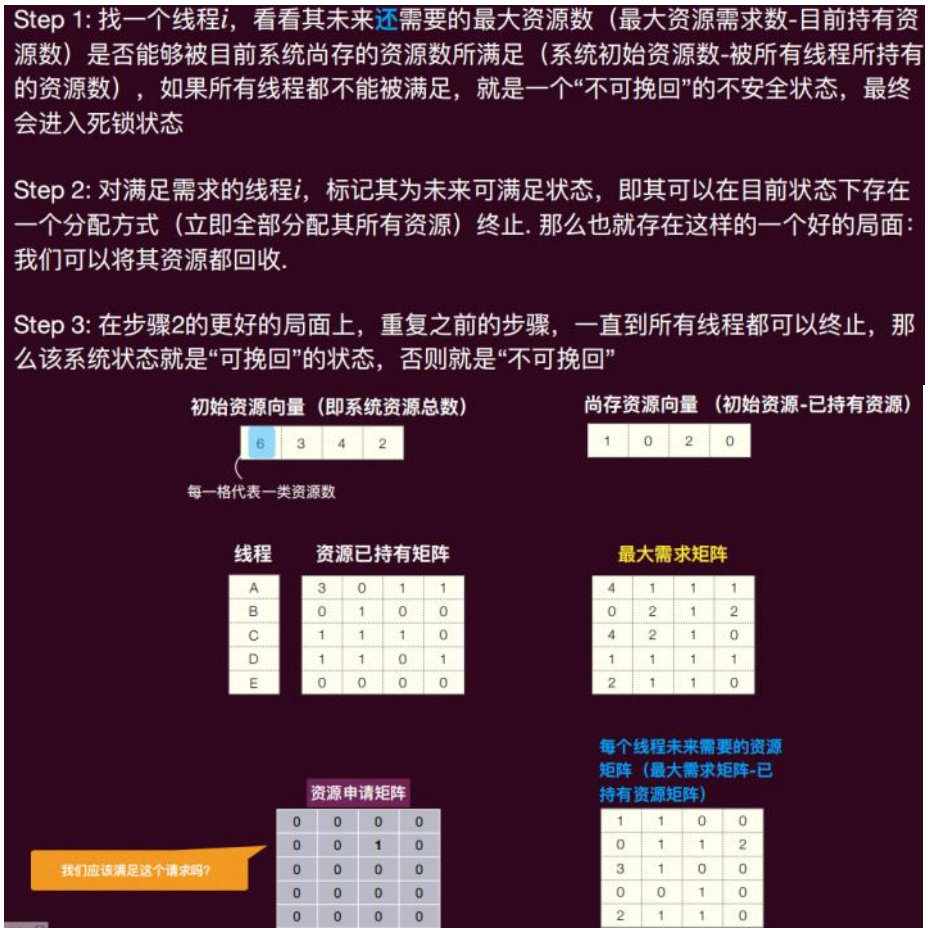
强制按照规定顺序申请锁 -> 有拓扑序意味无环

（六）不可挽回事件

在二维顺序图中，状态的移动方向只能是↗。



（七）银行家算法 —— 处理不可挽回的方法论



接受该资源申请后，存在一个方案使得所有线程都满足未来申请需求，因此该申请可以被接受！

然而，这个算法是不实用的

- 我们一般没法知道“最大”需求矩阵
- 系统的资源是动态变化的
  - 线程数会变化
  - 资源数可能会变化（线程可以自己释放所持有的锁）

（八）死锁的动态侦测 —— 让它发生吧

每次锁的acquire/release都记录 tid 和 lock name，动态构建锁的有向图 $G(V,E)$，并Assert该图没有环

- V：为每把锁的名字
- E：每个线程持有某把锁u之后再去尝试获得某把锁v，就加入边(u,v)
- 锁的名字可以用地址来唯一绑定 （也可以是锁所在的文件和行，这是一种近似，因为可能同一个行是malloc不同的锁，相应地，代价低）

如果资源不止一个？—— 类银行家

- 给定：所有线程的当前持有资源为矩阵C、所有线程的当前申请资源为矩阵R、以及系统资源总数向量E

系统资源总数E

6	3	4	2

尚存资源向量 (总数-持有数)

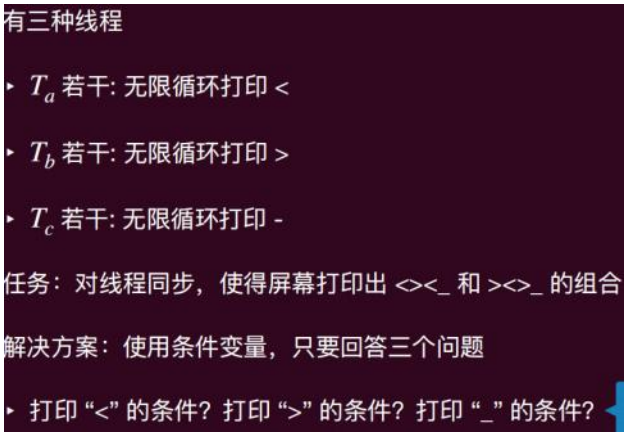
1	0	2	0

线程	资源已持有矩阵C				资源申请矩阵R			
A	3	0	1	1	2	0	0	0
B	0	1	0	0	0	0	1	1
C	1	1	1	0	0	1	0	0
D	1	1	0	1	0	1	1	0
E	0	0	0	0	2	0	0	0

- 按照和之前银行家算法类似的思路：
  - 找到可以满足要求的线程，先假定分配给其资源，然后释放其所有资源（创造更好的局面）
  - 不断重复上述步骤，直到所有线程都可满足即没有死锁
  - 否则就是出现死锁

题目

（一）打印fish




```c
#include "thread.h"
#include "thread-sync.h"
#define LENGTH(arr) (sizeof(arr) / sizeof(arr[0]))
enum { A = 1, B, C, D, E, F, };
struct rule {
  int from, ch, to;
} rules[] = {
    {A, '<', B},
    {B, '>', C},
    {C, '<', D},
    {A, '>', E},
    {E, '<', F},
    {F, '>', D},
    {D, '_', A},
};
int current = A, quota = 1;
mutex_t lk = MUTEX_INIT();
cond_t cv = COND_INIT();
int next(char ch) {
    for (int i = 0; i < LENGTH(rules); i++) {
        struct rule *rule = &rules[i];
        if (rule->from == current && rule->ch == ch) {
          return rule->to;
        }
    }
    return 0;
}
static int can_print(char ch) {
    return next(ch) != 0 && quota > 0;
}
void fish_before(char ch) {
    mutex_lock(&lk);
    while (!can_print(ch)) {
        cond_wait(&cv, &lk);
    }
    quota--;
```

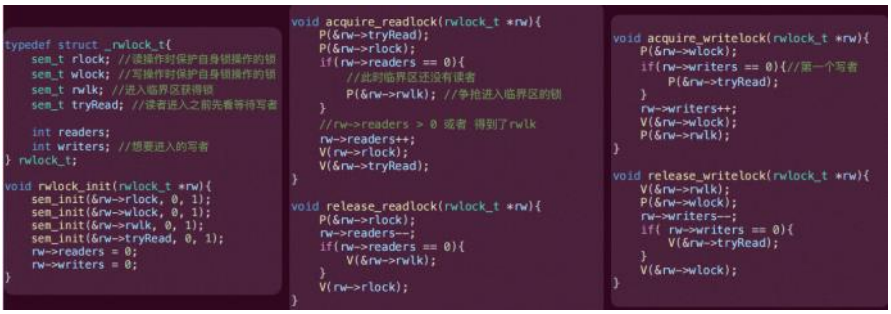
```
    mutex_unlock(&lk);
}
void fish_after(char ch) {
    mutex_lock(&lk);
    quota++;
    current = next(ch);
    assert(current);
    cond_broadcast(&cv);
    mutex_unlock(&lk);
}
const char roles[] = ".<<<<<>>>>___";
void fish_thread(int id) {
    char role = roles[id];
    while (1) {
        fish_before(role);
        putchar(role); // Not lock-protected
        fish_after(role);
    }
}
int main() {
    setbuf(stdout, NULL);
    for (int i = 0; i < strlen(roles); i++)
        create(fish_thread);
}
```

（二）读写锁

读优先



写优先



读写排队



（三）哲学家就餐

1. N-1 鸽巢原理
2. 拿最小编号的筷子
3. 碰撞 random 回退

虚拟化

进程和线程
上下文切换
fork、execve、exit
调度算法（round-robin、
FCFS、优先级、多级反馈队列）
优先级反转
虚拟内存
TLB
帧替换算法
多级页表

进程和线程

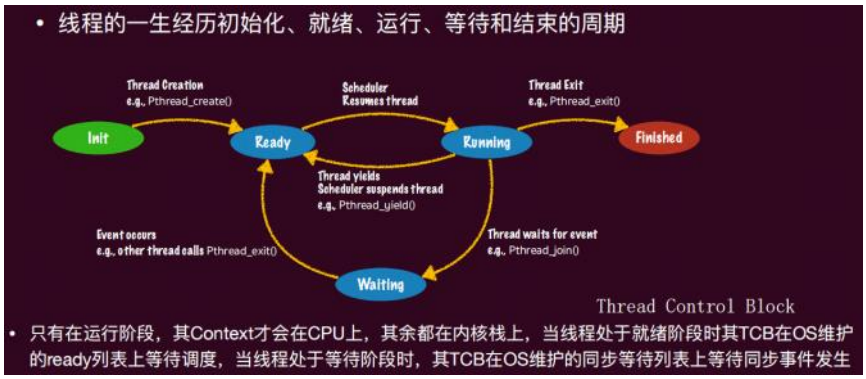
线程：

共享内存的执行流；

拥有独立的“上下文”和栈帧列表；

共享全局变量、堆空间。

TCB（线程控制模块）



进程：

拥有独立地址空间的运行实体，可以包含一个/多个线程

PCB（进程控制模块）

生命周期 和 线程很类似

不同系统的进程实现不同

（略）

进程树：

init (pid = 1)

终止后的进程的资源会被操作系统回收，并通知其父进程

i. why? 父进程有子进程的pcb，需要消除。
ii. 父进程调用wait系统调用会得到子进程的退出通知和退出状态，同时移除该子进程的PCB
iii. 僵尸进程：已终止 但 PCB信息未回收
iv. 孤儿进程：爸爸没了！Linux使用init接管

上下文切换

上下文切换（Context Switch）是操作系统中用于在多任务环境中管理进程或线程执行的一种技术。当操作系统决定将CPU从当前正在运行的进程或线程切换到另一个进程或线程时，就会发生上下文切换。

在上下文切换过程中，操作系统会执行以下步骤：

1. **保存当前上下文**：操作系统保存当前进程或线程的状态，包括程序计数器（指向下一条将要执行的指令）、寄存器状态、堆栈指针等。
2. **更新内存映射**：如果新进程或线程需要不同的内存空间，操作系统会更新内存管理单元（MMU）的映射，将新进程的虚拟地址映射到物理地址。
3. **更新CPU寄存器**：加载新进程或线程的寄存器状态，包括程序计数器和其他CPU寄存器。
4. **更新堆栈**：如果需要，操作系统会切换到新进程的堆栈。
5. **恢复新进程的执行**：操作系统恢复新进程的执行，从保存的程序计数器开始。

上下文切换的代价通常较高，因为它涉及到保存和加载大量的状态信息，以及可能的内存映射更新。频繁的上下文切换可能会导致系统性能下降，因为它减少了实际用于执行用户代码的CPU时间。

然而，上下文切换是多任务操作系统的关键特性之一，它允许多个进程或线程并发执行，提高了系统资源的利用率，并提供了更好的用户体验。操作系统会根据进程的优先级、系统负载和其他因素，智能地调度上下文切换，以实现高效的任务管理。

三类系统调用

fork 创建, execve 改变, exit 退出

不重要：

fork + wait()/waitpid(…) 等待子进程

1. **函数原型**：

```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *status, int options);
```

复制

2. **参数**：

- `pid`：指定要等待的子进程的进程 ID。可以使用负值、正值或特殊值：
  - `-1`：等待任何子进程。
  - 正整数值：等待特定子进程。
  - `0`：等待任何子进程，但只返回其进程组 ID 匹配调用进程的进程组 ID 的子进程。
- `status`：指向整数的指针，用于接收子进程的状态信息。
- `options`：指定等待调用的行为选项，常用的选项包括：

fork

fork

int pid = fork();

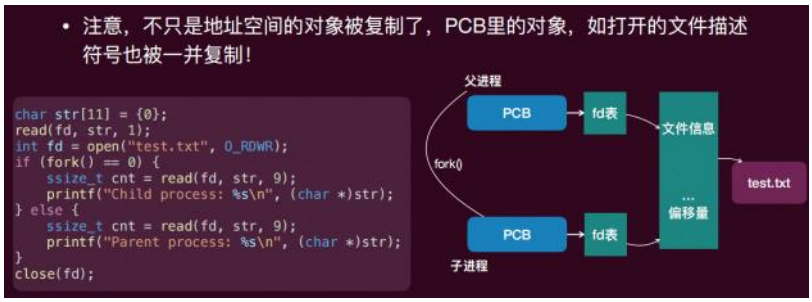
pid = 0: 子进程;

pid > 0: 父进程，pid = 子进程号;

pid = -1: 错误;

fork行为:

立即复制状态机



this is outputs:

```
mojo@Mojo-LAPTOPone:~/oslesson/lesson9/fork-demo$ ./forkfile
Parent process: bcdefghij
Child process: klmnopqrs
mojo@Mojo-LAPTOPone:~/oslesson/lesson9/fork-demo$ cat test.txt
abcdefghijklmnopqrstuvwxyzmojo@Mojo-LAPTOPone:~/oslesson/lesson9/fork-demo$ cat forkfile.c
```

特别注意！！！

fork() 还复制了**Buffer**!

```c
#include <unistd.h>
#include <stdio.h>

int main() {
    for (int i = 0; i < 2; i++) {
        fork();
        printf("Hello\n");
    }
}
```

```
mojo@Mojo-LAPTOPone:~/oslesson/lesson9/fork-demo$ ./forkdemo2
Hello
Hello
Hello
Hello
Hello
Hello
mojo@Mojo-LAPTOPone:~/oslesson/lesson9/fork-demo$ ./forkdemo2 | cat
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
```

为什么使用 | cat 会有 8 次输出?

可以通过理解进程分叉和缓冲之间的交互来解释实际观察到的 8 次输出行为。

缓冲行为

1. **缓冲机制**:
   - 当直接运行程序（./a.out）时，标准输出（stdout）是行缓冲的。
   - 当将输出通过管道（./a.out | cat）时，标准输出（stdout）是全缓冲的。
2. **分叉交互**:
   - 每个进程将有自己的缓冲 I/O。
   - fork() 之后，父进程和子进程都有自己的缓冲副本。如果在 fork() 后调用 printf，每个进程将独立打印。

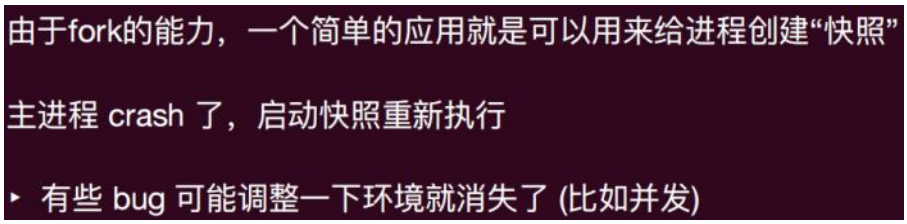
```
mojo@Mojo-LAPTOPone:~/oslesson/lesson9/fork-demo$ ./demo-2 | cat
Hello
Hello
Hello
Hello
Hello
Hello
mojo@Mojo-LAPTOPone:~/oslesson/lesson9/fork-demo$ cat demo-2.c
#include <unistd.h>
#include <stdio.h>

int main() {
```

```
mojo@Mojo-LAPTOPone: ~/oslesson/lesson9/fork-demo$ ./demo-2 | cat
Hello
Hello
Hello
Hello
Hello
Hello
mojo@Mojo-LAPTOPone:~/oslesson/lesson9/fork-demo$ cat demo-2.c
#include <unistd.h>
#include <stdio.h>

int main() {
    for (int i = 0; i < 2; i++) {
        fork();
        printf("Hello\n");
//      printf("Hello\t");
        fflush(stdout);
    }
}
```

fork的应用：



execve

- 很多时候，父子进程并不是同样的逻辑，子进程有自己的逻辑，比如shell创建进程并不是为了子进程也是一个shell，而是“加载”某个可执行文件进行运行
  - execve就是这样的系统调用
    - 其参数重，pathname是要加载的可执行文件具体的路径，argv进程执行所需要的参数（main函数同款参数），envp是

```
#include <unistd.h>
int execve(const char *pathname, char *const argv[], char *const envp[]);
```

一句话：将当前进程重置成一个可执行文件描述状态机的初始状态

- 加载pathname指定的可执行文件（数据段、代码段）
- 重新初始化堆和栈
- PCB中相应的memory mappings也会改变
- 将PC寄存器设置到可执行文件代码段定义的入口点，该入口点最终会调用main函数

execve调用成功时没有返回值，因为新进程映像覆盖了调用进程的映像。

A successful call to execve does not have a return value because the new process image overlays the calling process image

```
mojo@Mojo-LAPTOPone: /oslesson/lesson9/execve-demo$ ./demo
PWD=/home/mojo/oslesson/lesson9/execve-demo
HELLO=WORLD
SHLVL=0
_=/usr/bin/env
mojo@Mojo-LAPTOPone:~/oslesson/lesson9/execve-demo$ cat demo.c
#include <unistd.h>
#include <stdio.h>

int main() {
    char *const argv[] = {
        "/bin/bash",
        "-c",
        "env",
        NULL,
    };

    char *const envp[] = {
        "HELLO=WORLD",
        NULL,
    };

    // Reset the state machine to "/bin/bash"
    execve(argv[0], argv, envp);

    // We are here only on error.
    printf("Hello, World!\n");
}
```

execve不会改变PCB中的文件描述符，那些打开的文件描述符还会保持打开

这其实很方便

- 比如shell里开启的进程打印字符都会在同样的终端(继承了stdout所指向的相同的tty)

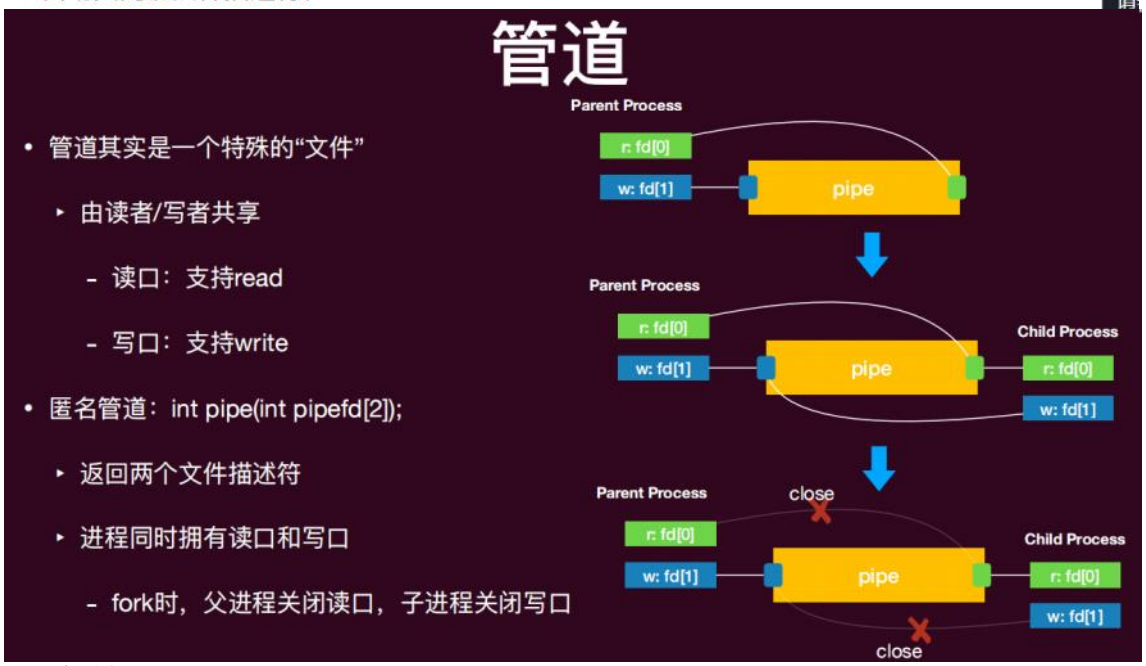
- 比如管道的实现

但如果是父进程打开了一个普通文件，在地址空间里有一个相应的file变量索引，但你的子进程重置了整个地址空间，因此那个文件描述符对应的file变量也没有了，此时你无法close这个文件了！

- 这会造成资源的泄漏，当然这些资源（PCB）都会随着进程的终止而最终被回收，但在运行期间还是有一些资源的损耗

- 可以在创建文件时增加一个选项 FD_CLOEXEC：close on exec

1. **父进程打开文件**：当一个父进程使用 `open` 系统调用打开一个文件时，它会获得一个文件描述符（通常是一个非负整数），这个文件描述符与内核中的 `file` 结构相关联，用于访问文件数据。
2. **子进程的地址空间重置**：当父进程调用 `fork` 创建子进程时，子进程会获得父进程地址空间的副本。如果子进程随后调用 `exec` 系列函数（如 `execve`）来执行一个新的程序，子进程的整个地址空间将被新程序的代码和数据覆盖。
3. **文件描述符和 `file` 变量的关系**：文件描述符是一个用户空间的抽象，它指向内核中的 `file` 结构。当子进程的地址空间被重置后，原有的 `file` 结构指针不再有效，因此无法通过原来的文件描述符引用来关闭文件。
4. **资源泄漏问题**：由于子进程无法关闭从父进程继承来的文件描述符，这可能导致文件描述符的泄漏。虽然进程终止时，其资源会被操作系统回收，但在进程运行期间，未关闭的文件描述符是一种资源浪费。
5. **解决方案 - FD_CLOEXEC 标志**：为了避免这个问题，可以在使用 `open` 或 `fcntl` 系统调用创建或修改文件描述符时，设置 `FD_CLOEXEC` 标志。这个标志指示操作系统，在执行 `exec` 系列函数时自动关闭该文件描述符。



COW 机制：

- 在某些系统中，execve 可能会利用写时复制（Copy-On-Write，COW）机制，以减少内存复制的开销。

exit

- void exit(int status) 是C标准库函数，也是最常用的进程退出函数:
  - 调用atexit()注册的函数；使得我们可以指定在程序终止时执行自己的清理动作。(atexit()最多可以注册32个函数，调用顺序与注册顺序相反)
  - 关闭所有打开的流(stdio)，这将导致写所有被缓冲的输出
  - 移除所有的临时文件
  - 最后调用_exit()函数终止进程

```
#include <stdlib.h>
void exit(int status);
```

The exit() function never returns

1. 正常退出

- 从main函数返回是最常见的终止方式
- main函数的返回值和调用exit()的传入参数int status是同样的语义
  - 0是函数是符合预期终止，非0是函数出现了错误终止

```
#include <stdlib.h>
int main(int argc, char **argv) {
  /* ... */
  if (/* Something really bad happened */) {
    return EXIT_FAILURE;
  }
  /* ... */
  return EXIT_SUCCESS;
}
```

很多编译器实现main的返回是类似如下这种方式

```
void _start(void) {
  /* ... */
  exit(main(argc, argv));
}
```

_exit()函数行为:

- 所有属于这个进程的文件描述符都会被关闭
- 所有该进程的子进程都会被init进程接管
- 向该进程的父进程发送SIGCHLD信号，通知改父进程其已经终止
- 注意：_exit()只会终止当前的线程，但libc做了一层wrapper，其实真实调用exit_group()，关闭所有线程

直接调用syscall(SYS_exit, 0);可以绕过这个wrapper

```
#include <unistd.h>
void _exit(int status);
```

2. 异常退出

- abort()系统调用会导致系统异常终止
  - atexit注册的函数不会调用
  - io流不会关闭

- abort()系统调用会导致系统异常终止
  - atexit注册的函数不会调用
  - io流不会关闭
  - 其行为就是产生一个SIGABRT信号发送给调用abort()的进程，然后改进程就会异常终止
    - 不会被ignore掉

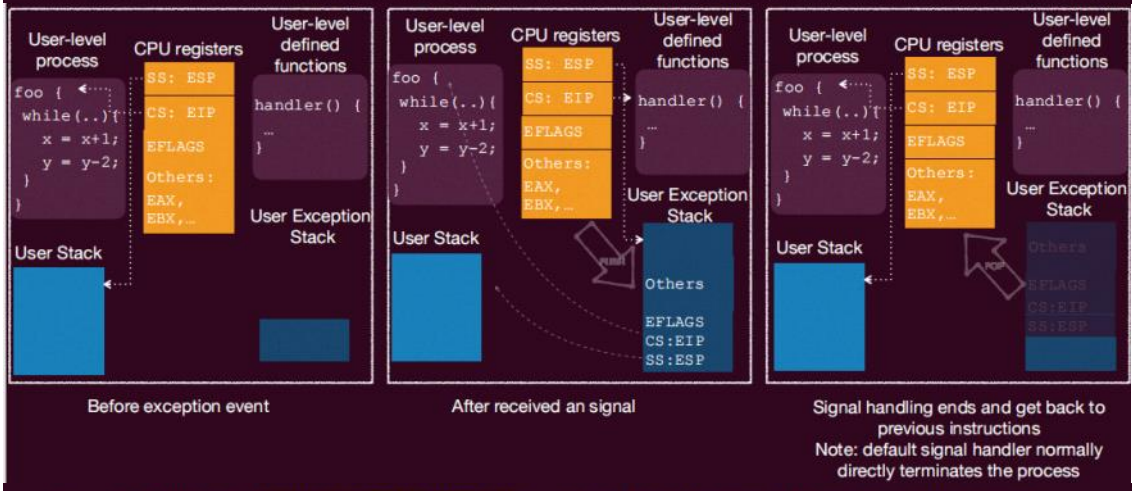
```
#include <stdlib.h>
void abort(void);
```

Signal

Linux操作系统提供了一种可以通知进程发生了某个事件的机制：信号（Signal），注：不要和并发的singal原语混淆，也不要和“信号量”（Semaphore）混淆

其本质上是对中断的模拟，命令`kill -l` 可以查看有多少信号， man 7 signal查看信号的具体信息

- 发生某个中断事件之后，用户程序也想获知并处理（而不只是在内核态由操作系统透明的处理）



- signal函数可以注册信号处理函数，只要传入相应的信号和函数名即可
  - handler设为SIG_IGN时表示忽略这个信号，比如signal(SIGCHLD,SIG_IGN) 就表示忽略掉子进程的终止信号，此时子进程结束会直接被内核完全清除（而不必先变为僵尸进程，然后再被回收）
  - handler设为SIG_DFL时表示采用linux默认的处理函数

```
#include <signal.h>
signal(SIGNAL_NUMBER, handler);
```

在 Linux 中，signal函数对同一信号的多次调用会导致先前的信号处理函数被新的调用覆盖

有了信号机制，可以完成很多异步的操作：

- 比如signal(SIGCHLD， handler)，可以在handler里进行wait，而不是在main函数里wait/waitpid从而阻塞父进程
- 有比如signal(SIGIO, handler)可以不用等待I/O完成，可以先做其他事情，如果文件描述符所指向的数据传输完成，会产生SIGIO的事件，就可以通过回调函数handler来处理

信号也是一种进程间通信（Inter-Process Communication, IPC）的机制之一

- 此外还有消息传递、共享内存、管道等

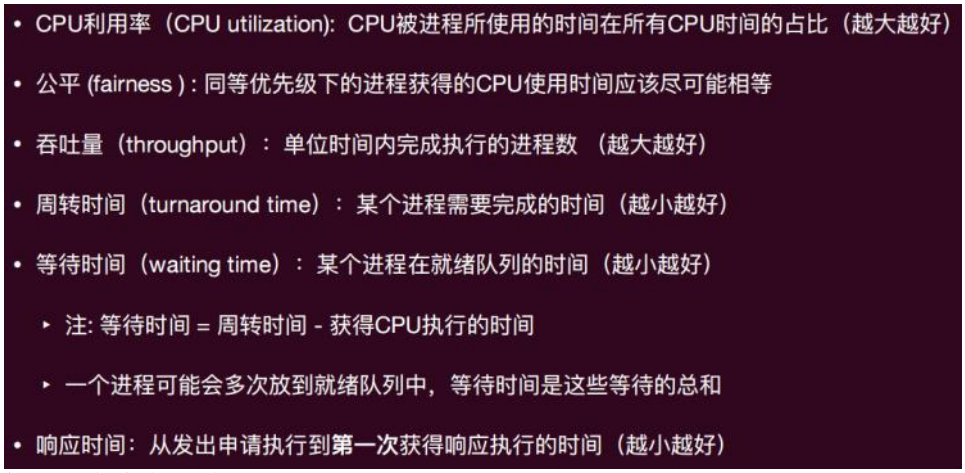
在 Linux 系统中，信号是进程间通信的一种形式，用于通知进程发生了某些事件。以下是一些常见的信号：

1. SIGINT：中断信号，通常由用户通过 Ctrl+C 产生，用于请求进程终止。
2. SIGTERM：终止信号，由操作系统发送，用于请求进程平滑地终止。
3. SIGKILL：杀死信号，不能被进程捕获或忽略，用于强制立即终止进程。
4. SIGSTOP：停止信号，不能被捕获、忽略或阻塞，用于暂停进程的执行。
5. SIGCONT：继续信号，用于继续被停止的进程。
6. SIGCHLD：子进程终止或停止信号，当一个子进程终止或停止时发送给父进程。
7. SIGALRM：由计时器到期后发送的信号，用于通知进程计时器超时。
8. SIGUSR1 和 SIGUSR2：用户自定义信号，可以由程序员定义用途。
9. SIGFPE：浮点异常信号，如除以零、溢出等。
10. SIGSEGV：段错误信号，当进程访问无效的内存段时发送。
11. SIGPIPE：管道信号，当进程尝试写入一个已经被读取端关闭的管道时发送。
12. SIGQUIT：退出信号，通常由用户通过 Ctrl+\ 产生，进程终止并生成核心转储。
13. SIGHUP：挂起信号，通常在终端类型改变时发送，也可用于重启进程。
14. SIGABRT：由 abort 函数生成的信号，用于异常终止进程并生成核心转储。
15. SIGTRAP：陷阱信号，由断点指令或调试器产生。
16. SIGIO：I/O 信号，当进程感兴趣的 I/O 操作准备就绪时发送。
17. SIGURG：紧急信号，当套接字上有紧急数据时发送。
18. SIGXCPU：CPU 时间限制超标信号，当进程超出了其 CPU 时间限制时发送。
19. SIGXFSZ：文件大小限制超标信号，当进程试图扩大文件超出其文件大小限制时发送。
20. SIGWINCH：窗口大小改变信号，当终端窗口大小改变时发送。

这些信号可以通过系统调用如 kill 显式发送，或在特定条件下由操作系统隐式产生。进程可以通过注册信号处理函数来捕获和响应这些信号。

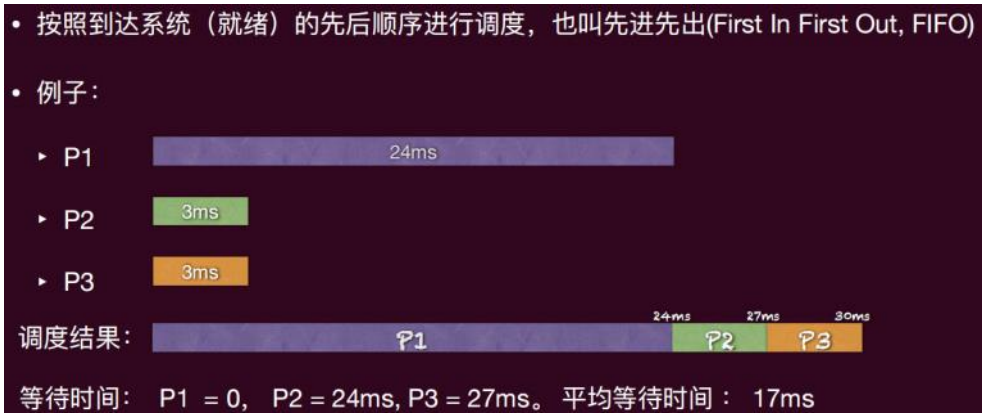
调度算法(紫色粗体字为重点)

0. 调度指标：

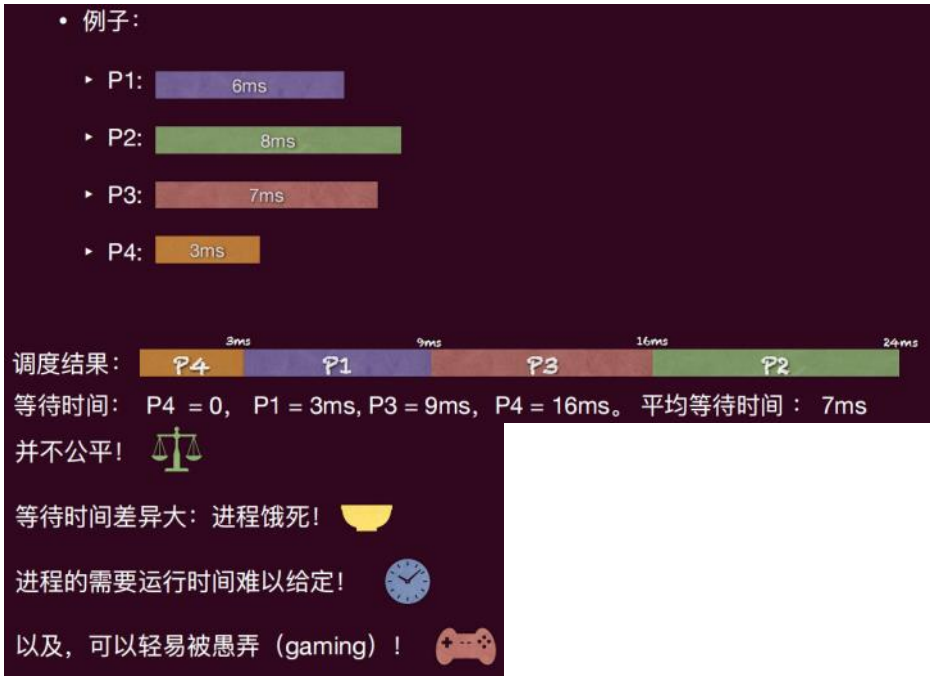


1. **批处理任务的调度：** 批处理任务通常是指不需要即时响应的作业，它们可以批量提交给系统进行处理。这类任务往往对时间要求不严格，更注重**吞吐量，即单位时间内完成的任务数量**。批处理调度策略通常优化作业的执行效率，减少I/O操作和CPU的空闲时间，从而提高系统的整体效率。

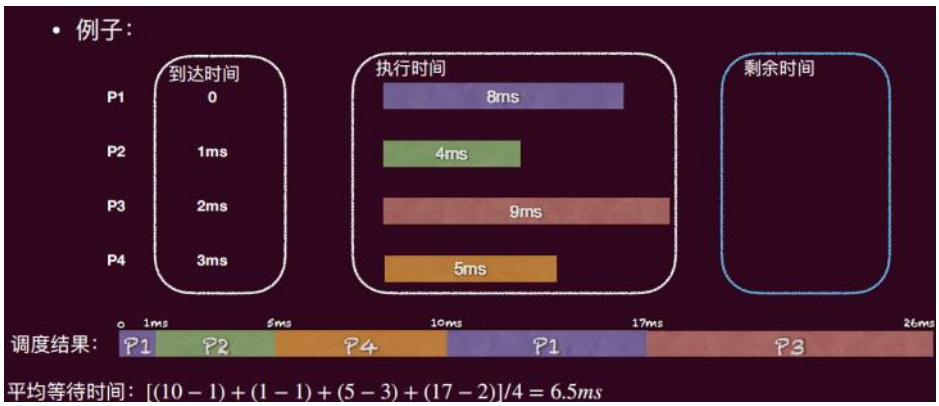
1.1 FCFS（先来先服务）—— 护航效应



1.2 SJF（最短任务优先）



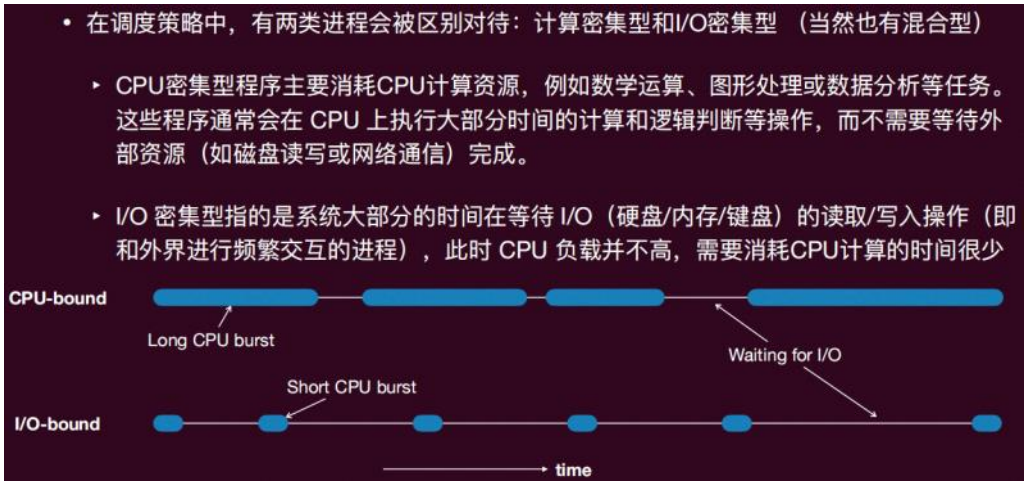
1.3 SRTF（最短剩余时间优先）—— **抢占式算法**



2. **交互性任务的调度：**

需要**快速响应用户输入**，例如文字处理、网页浏览或数据库查询等。

对**响应时间**有较高要求，因此调度策略需要保证这些任务能够得到及时的处理。



2.1 Round-Robin（时间片轮转调度）



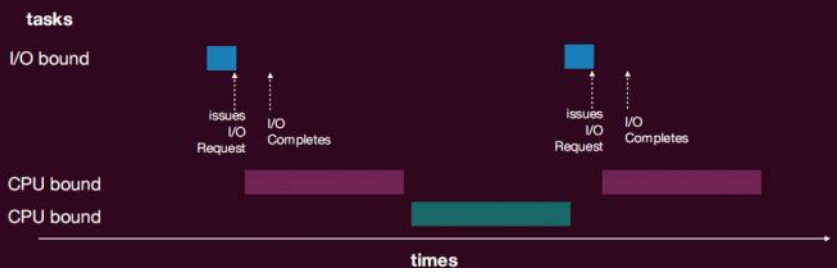
问题：

a. 更长的平均周转时间，并且有更高的开销！

b.

在运行既有 I/O 密集型任务又有计算密集型任务的情况下，当 I/O 密集型任务执行 I/O 操作时，它会让出处理器（由于需要的CPU计算很短，没有用完时间片就让出CPU了）。

- 这时即使 I/O 操作很快完成（比如你正在用VIM打字），也必须等待重新分配处理器，直到其他计算密集型任务用完他们完整的CPU切片。

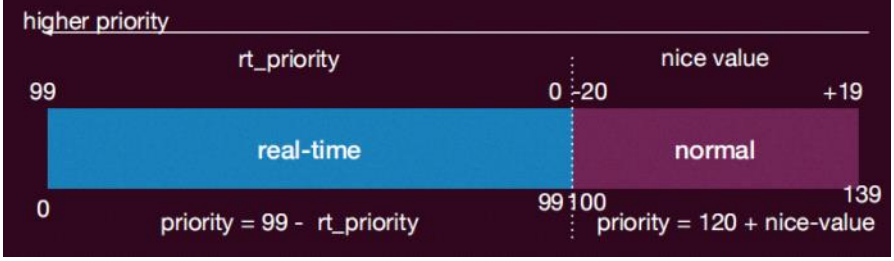


这种情况可能导致 I/O 密集型任务的响应性降低

2.2 基于优先级的调度

I/O-bound和CPU-bound的优先级不同，我门需要基于优先级的调度：

- 每个进程都关联有一个优先级数（整数）。
- 每次发生调度时，CPU被分配给具有最高优先级的进程。
- Linux下默认的优先级(转换之后)：
  - 实时任务(real time): 高优先级 (0 - 99)
    - 试试`chrt`命令：chrt -r -p priority PID
  - 普通任务(normal ones): 低优先级 (100 - 139)
    - 试试`renice`命令 renice [-n] priority [-g|-p|-u] identifier
    - 试试`nice`命令 nice [option] [command [ARG]...]



问题：

饿死(Starvation) – 低优先级的进程可能永远不会执行。

- 解决方案： 老化（Aging)–随着时间的推移增加进程的优先级。
  - 需要一种能动态改变优先级的策略，如：多级反馈队列（**multilevel feedback queue**）

2.3 MFQ 多级反馈队列

典型模式：

- 一组轮转队列：
  - 每个队列都有单独的优先级。
- 高优先级队列拥有短的时间片。
  - 低优先级队列拥有长的时间片。
- 调度器选择最高优先级队列中的第一个进程。
- 进程加载到内存中时初始在最高优先级队列中。
- 如果时间片到期，任务会降低一个级别。

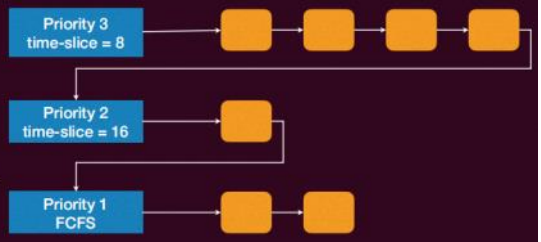
例子：

例子： 三个优先级队列：

- Priority 3：轮转调度，时间片为8毫秒
- Priority 2：轮转调度，时间片为16毫秒
- Priority 1：先进先出（FCFS）可视为时间片无穷大

调度过程如下：

- 新进程进入队列Priority 3（最高优先级），以轮转调度的方式服务。
- 当获得CPU时，该进程获得8毫秒的时间片。
- 如果在8毫秒内未完成，则将进程移动到队列Priority 2。
- 在Priority 2中，作业再次以轮转调度的方式服务，并获得额外的16毫秒。
- 如果仍然没有完成，则被抢占并移动到队列Priority 3，进行FCFS调度



效果：

当使用多级反馈队列（MFQ）时：

- CPU密集型进程将下沉到长时间片的优先级队列。
  - 如果使用完时间片，进程会下降一个优先级。
  - 较大的时间片可以减少上下文切换的开销。
- I/O 密集型进程将保持在高优先级队列中。
  - 如果一个进程没有完成其时间片（即，它在 I/O 操作上被阻塞），那么它将保持在相同的优先级水平。

成绩&问题

多级反馈队列仍然存在饥饿问题：

- 如果有大量交互式进程或者频繁创建新进程，则高优先级队列中始终有可用任务。此时，低优先级队列中的 CPU 密集型进程将永远不会被调度。

一个相关的问题是，一个交互式进程可能最终会处于低优先级水平。

- 如果一个进程的某个时期变得 CPU 密集型，它就会降到低优先级水平，而且注定会永远留在那里。一个例子是一个游戏需要花费大量 CPU时间来初始化，但随着初始化完成，就变为了等待玩家输入的交互式进程 （此调度下的游戏体验可想而知）

因此，MFQ需要一个策略（老化）来定期增加进程的优先级，以确保它会被调度运行。一个简单的方法是定期将所有进程提升到最高优先级队列，即重置

MFQ调度器可以在周转时间、低开销和公平性之间取得平衡。 事实上，它被大多数商业和现实操作系统（包括Windows、MacOS和Linux）使用（作为一种策略）

但这个MFQ还是可以被愚弄的

- 比如一个恶意攻击者了解调度器的工作原理，那么他/她可以编写代码，在时间片到期之前强制系统在某些低延迟的 I/O 操作上阻塞（例如，睡眠几毫秒）
- 这样，即使进程反复消耗大部分时间片，它也会因为没有使用完时间片而受到奖励（不会被降低优先级），从而始终保持在高优先级水平。

方法论：

解决方案 — 追踪：

- 不仅仅是简单地检查进程是否使用完了它的时间片，调度器会跟踪进程在较长时间间隔（几个时间片）内运行的总时间。
- 每个优先级队列将有一个与之关联的最大CPU时间分配。
  - 超出实用的分配则降低优先级

2.4 Lottery（乐透）

一个灵活的调度算法：乐透算法

- 给每个作业分配一定数量的彩票票数
- 然后随机选择一个中奖票（拥有更多票数的作业有更大的中奖机会）。
- 为了避免饥饿，至少给每个作业分配一张彩票。

3. **实时任务的调度：** 本门课只考虑简单的固定周期任务。

   对**任务完成时间有严格限制**的作业；截止日期以规律的间隔发生。

3.1 RM 单调速率调度（Rate Monotonic）：

前置条件：处理时间/周期 的 和 小于等于 1。

根据其速率（即周期的倒数）分配优先级。

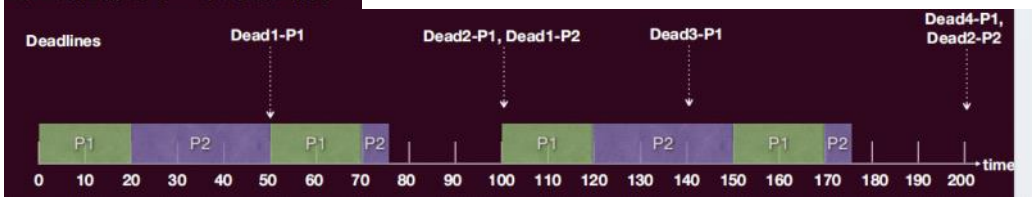
- 周期较短的任务具有较高的优先级，
- 而周期较长的任务具有较低的优先级。
  - 例如，有两个进程P1和P2，它们的周期分别是50和100，那么P1首先被调度，然后是P2，P1可以抢占P2。

这种策略背后的理念是为需要更频繁占用CPU的任务分配更高的优先级。

P1的周期为50，处理时间为20，

P2的周期为100，处理时间为35，

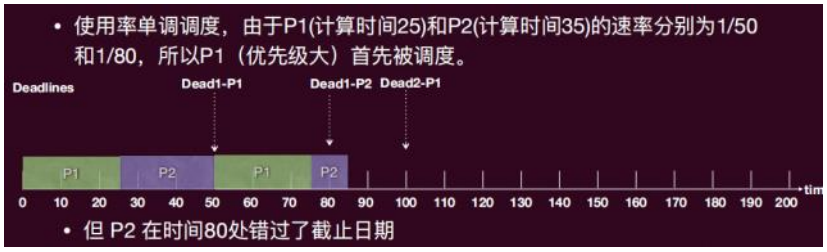
截止期限都为下一个周期之前。



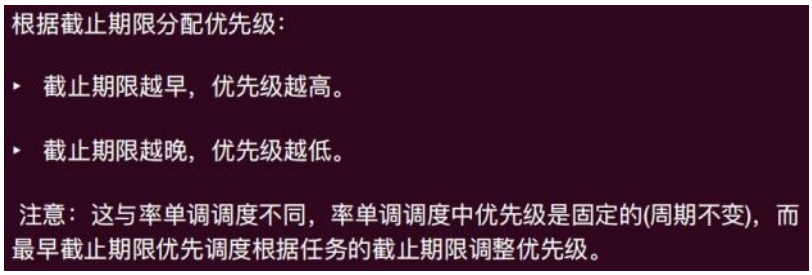
大失败：

P1的周期为50，处理时间为25，截止期限为下一个周期之前。

P2的周期为80，处理时间为35，截止期限也是在下一个周期之前。

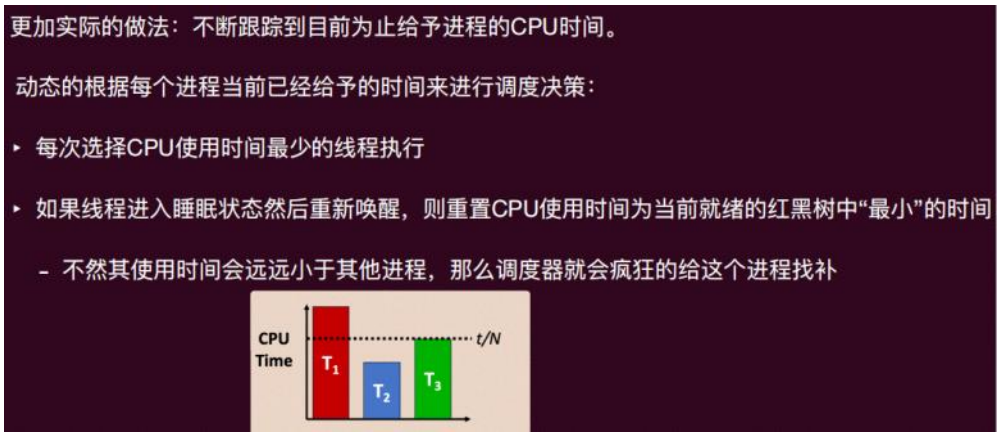


3.2 EDF 最早截止日期优先



4. **真实的调度**

4.1 完全公平调度策略



如何找到最小：红黑树！

如何体现“更加公平？”：增加时间的权重！

5. **总结：**

每一类任务的调度策略都有其特定的目标和优化方向，操作系统会根据运行的任务类型选择合适的调度策略，以满足不同任务的需求。例如，在需要处理大量用户请求的服务器上，可能需要更注重交互性任务的响应速度；而在工业控制系统中，则可能需要优先保证实时任务的调度。

6. 补充：

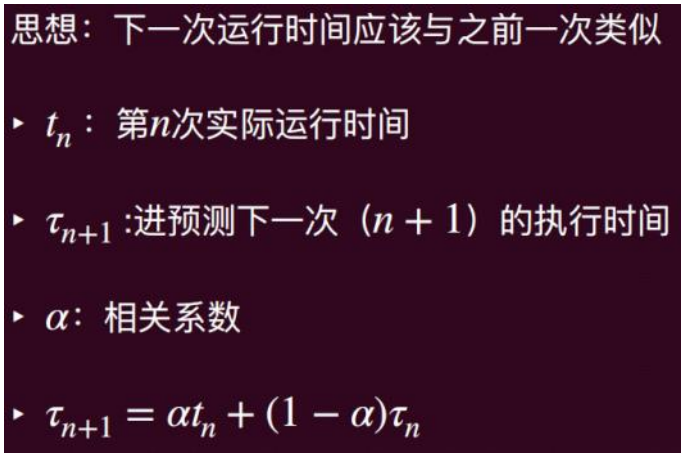
EMA（时间预测算法 exponential moving average) 指数平移

在操作系统中，调度算法负责决定哪个进程应该获得 CPU 的使用权。虽然指数移动平均（EMA）本身是一种用于时间序列分析的统计方法，它并不直接关联到操作系统调度，但是我们可以借鉴 EMA 的思想来设计或改进调度算法。以下是一些可能的联系：

1. **动态优先级调整**：操作系统可以根据进程的历史行为（如 CPU 使用时间）动态调整其优先级。EMA 可以用来平滑地计算进程的 CPU 使用率，从而为优先级调度提供依据。
2. **自适应调度**：操作系统可以利用 EMA 来实现自适应调度策略，即根据进程的当前行为和历史表现来调整调度策略，例如，给予最近表现良好的进程更高的 CPU 分配权重。
3. **负载预测**：操作系统可以使用 EMA 来预测系统的负载变化，从而更好地分配资源和调整调度策略，以应对高负载或低负载的情况。
4. **响应时间估计**：通过分析进程的 CPU 请求历史和响应时间，EMA 可以帮助操作系统估计进程的响应时间，进而优化调度决策。
5. **公平性保证**：EMA 可以帮助操作系统实现公平调度，通过平滑地考虑所有进程的 CPU 使用历史，确保每个进程都能获得合理的 CPU 时间。

6. **实时调度**：在实时系统中，操作系统可以使用 EMA 来监控和预测任务的执行时间，以确保满足实时约束。
7. **资源分配**：操作系统可以利用 EMA 来平滑地估计进程或任务的资源需求，进而在多任务环境中合理分配 CPU、内存等资源。
8. **性能监控**：操作系统可以使用 EMA 来监控系统性能指标，如 CPU 使用率、内存使用率等，以便及时调整调度策略和资源分配。

尽管 EMA 本身不是操作系统调度算法，但其平滑和加权平均的思想可以应用于设计更智能、更适应性强的调度策略。这种应用需要创造性地将统计方法与操作系统调度的需求结合起来。



优先级反转——并发+调度的Bug

假设系统中有两个线程：T1（低优先级）和T2（高优先级）。 如果两者都是可运行的，调度器将始终优先运行T2。

现在假设T1首先运行（T2此时还不可运行），并获得了一个自旋锁，并进入其临界区。

然后T2开始运行，调度器立即调度T2取代T1运行；

但T2尝试获取锁，并开始忙等待。 由于T2运行时永远不会调度T1，因此T1没有机会离开临界区，而T2则永远循环下去。

一个高优先级任务反而受制于一个低优先级任务。

- 只要低优先级任务持有锁，高优先级任务就会被阻塞。

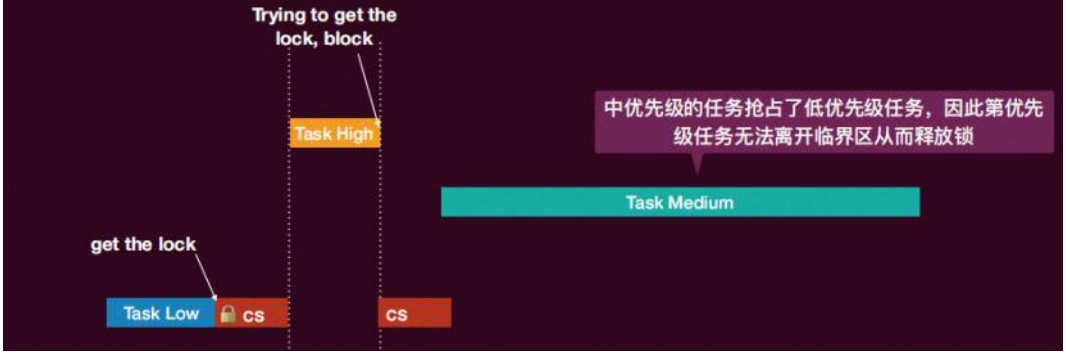


然而，即使避免使用自旋锁，而是使用futex wait的阻塞也并不能避免优先级反转问题。

假设有三个线程，T1、T2和T3，其中T3的优先级最高，T1的优先级最低。

- T1获取了一个锁，T3开始运行，从而抢占了T1。
- T3试图获取锁，但被阻塞。
- T2开始运行，因为它的优先级高于T1，所以它将运行。
- T3的优先级高于T2，但由于T2正在运行，T3被阻塞在等待T1上，而现在T1可能永远不会运行。

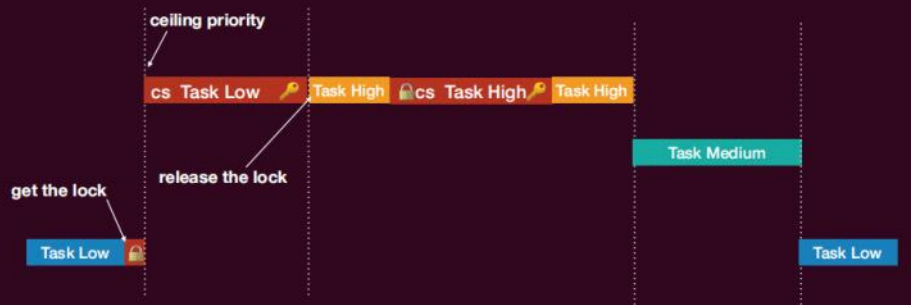
一个中优先级任务在持有锁时中断了低优先级任务的执行。



方法论：

优先权极限（priority ceiling protocol）

- 每当一个任务获取一个锁时，该任务的优先级被提升到与该锁关联的优先级上限相同的优先级。



优先级继承

- 当一个任务持有一个锁时，如果其他（更高优先级的）任务试图获取该锁，那么持有锁的任务的优先级将被提升到那个更高优先级的任务的优先级。



虚拟内存

虚拟地址 好处：

每个应用程序拥有独立的虚拟地址空间

- 应用程序认为自己独占整个内存(透明性)
- 应用程序不再看到物理地址
- 应用加载时不用再为地址增加一个偏移量

优点，可以很自然地提供：

- 保护
- 重定位
- 数据共享
- 连续空间假象

问题：

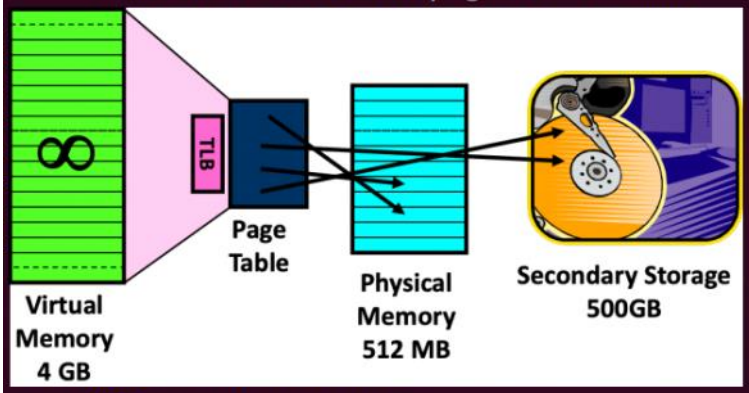
连续内存分配

多个进程时需要将物理内存进行分割，每个进程占据一个连续的物理内存分区，有两种分割方式

- 固定分区（Fixed-Partition）
  - 物理内存一开始就被分为“固定”大小的区域，各个区域的大小可以相同，也可以不同，一个进程选择一个空闲区域进行分配
- 可变分区（Variable-Partition）
  - 物理内存区域大小和数量是可变的，根据进程的具体需要分配相应的空间
    - 需要维护一个空闲的内存集合，开始是整个巨大的空间，但随着进程的分配和回收，内存会存在很多大小不一的空闲的“孔”

虚拟内存

- 每个进程都有一个大地址空间的幻觉。
- 支持多个并发运行的进程使用大虚拟地址空间：只将常用的页面保留在内存中（此时内存可以看成是所有pages的一个cache）



- 透明的间接层（页表）
  - 支持物理数据的灵活放置
    - 数据可以在磁盘上甚至是网络中的某处
  - 数据位置的变化对用户程序是透明的

具备“部分”加载程序的执行能力

- 程序不再受物理内存限制（可以运行无法完全放入物理内存的程序）
- 每个程序在运行时占用更少的内存（可以同时运行更多的程序）
- 加载或交换程序到内存中所需的I/O更少（每个程序启动更快）

使用存在/不存在位（在页表项中）来跟踪哪些页面存在于物理内存中。

当程序引用其地址空间的一部分时：

- 如果页面在物理内存中，则直接进行地址转换
- 如果不在，则发生**缺页异常（Page Fault）**，操作系统被调用来处理该异常：
  - 检测并将页面加载到内存中，然后重新执行指令(引用该地址空间的指令)

缺页处理：

1. 硬件陷入内核，进行Context-Switch（将程序计数器保存在栈上，保存通用寄存器和其他易失性信息。）

2. 系统发现了这个事件是一个缺页异常，尝试确定所需的虚拟页面。

3. 一旦知道引发缺页异常的虚拟地址，系统检查地址是否有效，并且保护是否与访问一致。

4. 找到一个空闲（干净的）帧。

- 如果没有空闲帧，则运行页面置换以选择一个victim(受害者)。
- 如果所选帧是脏的，则将页面安排转移到磁盘，进行上下文切换，暂停引发异常的进程。

5. 一旦帧变为干净状态，系统查找所需页面的磁盘地址，并安排磁盘操作将其调入（引发缺页异常的进程仍处于暂停状态）

6. 当磁盘中断指示页面已经到达时，更新页表，并将帧标记为正常状态

7. 将引发缺页异常的指令恢复到其原始状态，并重置程序计数器

8. 引发缺页异常的进程被调度，Context-Switch回去

性能： 有效访问时间（Effective Access Time, EAT）

内存访问时间 = 200 ns

平均缺页异常服务时间 = 8 ms

EAT = (1 – p) x 200 + p x 8,000,000 = 200 + p x 7,999,800 ns

如果 1,000 次访问中有一次引发缺页异常 (p = 0.001), 那么 EAT = 8200 ns (减速了 40 倍)

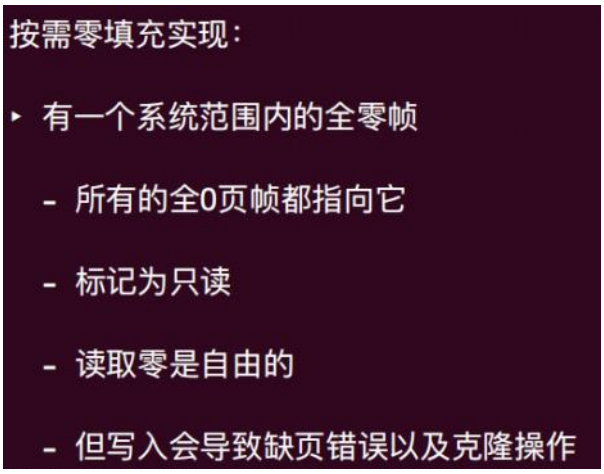
如果想要性能降级 < 10%

- 200 + p x 7,999,800 < 220
- p < 0.0000025 (每 400,000 次内存访问中不到一次缺页异常)

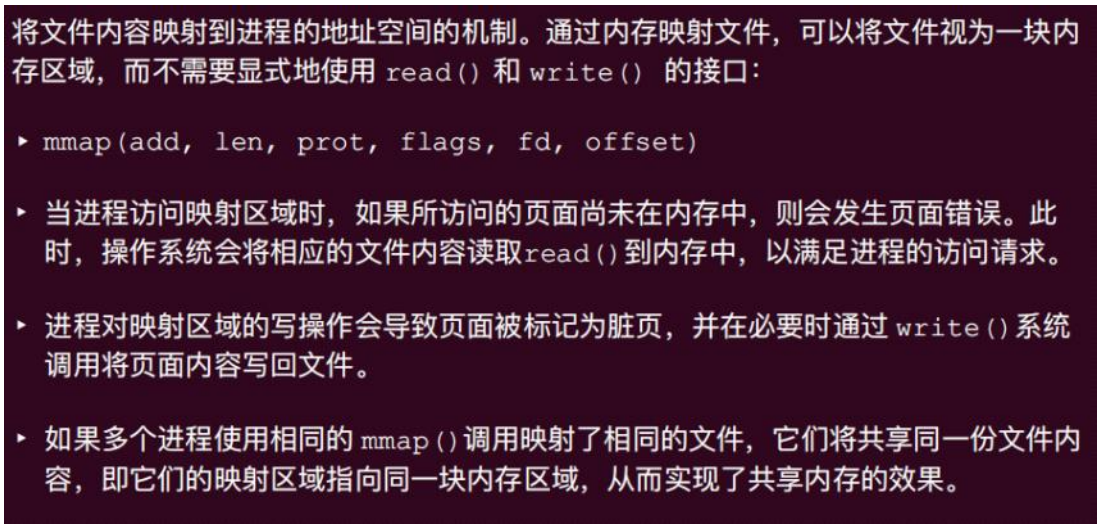
应用：

写时复制

零 页



MMF 内存映射文件（Memory-Mapped Files）



a. addr：内存映射的起始地址。如果指定为 NULL，则系统会选择合适的地址来创建映射，否则系统会尝试在指定的地址创建映射。
b. len：映射区域的长度。这个长度必须与文件大小或者页的大小（通常是4KB）对齐。
c. prot：映射区域的保护方式。可以是以下几种方式的组合：
   - PROT_EXEC：允许执行映射的内存。
   - PROT_READ：允许读取映射的内存。
   - PROT_WRITE：允许写入映射的内存。
   - PROT_NONE：不允许任何操作。
d. flags：控制映射行为的标志位。常见的标志位包括：
   - MAP_SHARED：对映射区域的修改会反映到文件上，同时允许其他映射了同一文件的进程看到这些变化。
   - MAP_PRIVATE：对映射区域的修改不会反映到文件上，是私有的copy-on-write映射。
   - MAP_ANONYMOUS：创建匿名内存映射，不与任何文件关联。
   - MAP_FIXED：强制在指定的地址创建映射。
e. fd：文件描述符。如果使用匿名映射，这个值应该是 -1。
f. offset：文件映射的偏移量，通常应该是页大小的整数倍。

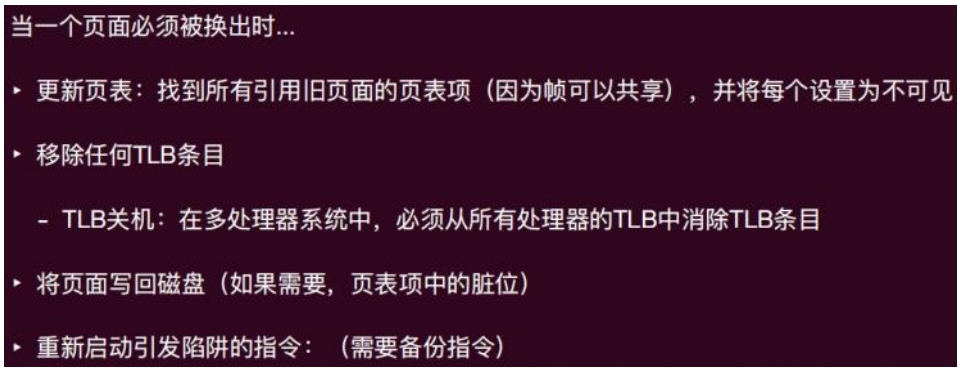
帧替换算法/页替换算法

页表项（PTE） 控制位

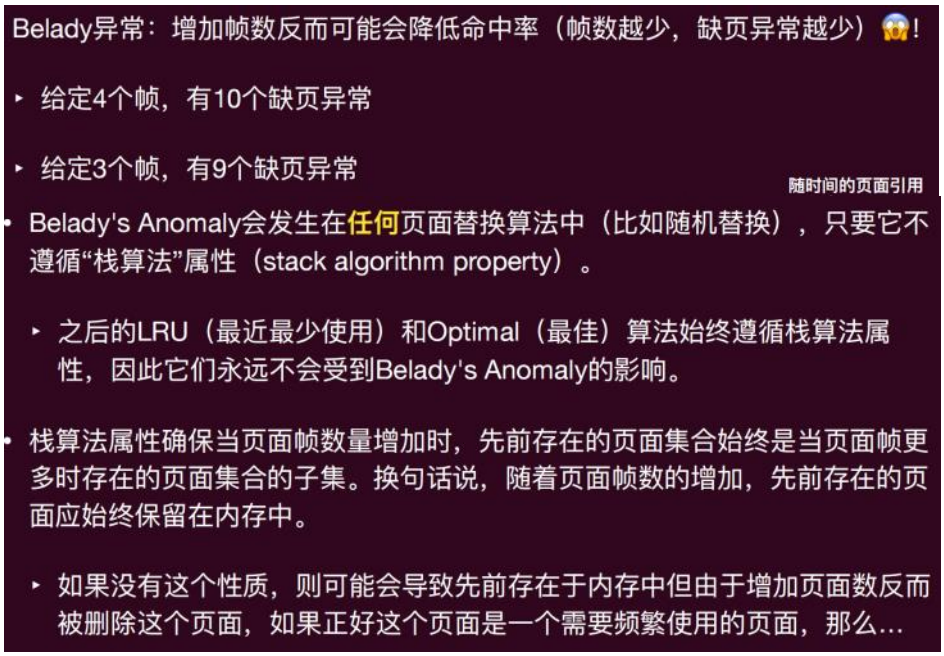


引用位 又称 访问位、参考位

页面置换行为：



FIFO - 先入先出



Optimal - 最长时间不会被使用



5替换的逻辑：

编号	1	2	3	4
未来多长时间不会被使用	1	2	3	4

LRU - 最近最少用

• 替换那些在最长时间内没有被使用的页面



5替换的逻辑：

编号	1	2	3	4
过去多长时间没有被使用	2	1	4	3

近似LRU - 二次机会算法

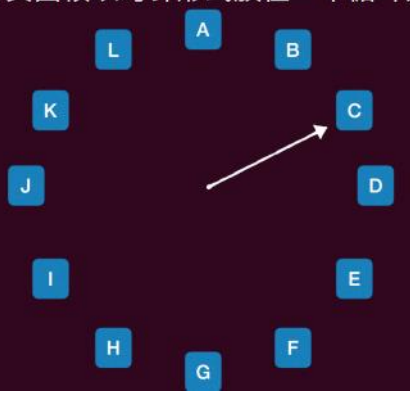
通过引用位来近似LRU（即二次机会算法，Second Chance）：寻找一个在最近的时钟周期内没有被引用的老页面。

- 系统中每个页面有一个引用位（R）。
- 每当引用页面（即读取或写入），引用位被设置为1（由硬件完成）。
- 如果要被替换的页面：
  - R = 1：将引用位设置为0；将其放在FIFO队列的末尾；并检查下一个页面。
  - R = 0：替换它。
- 如果所有页面都被引用，那么第二次机会等于FIFO。

实现改进：

高效替代Second Chance算法：时钟算法

- 将所有页面帧以时钟形式放在一个循环列表中（避免在队列上移动页面）



发生缺页异常时，检查当前指向的页面。
采取的操作取决于R位的值：
- R = 0：驱逐该页面
- R = 1：将R=0并将指针向前移

N 次 机会算法

第N次机会算法：给每个页面N次机会

- 操作系统为每个页面保持一个计数器：计数扫过的次数
- 发生页面错误时，操作系统检查使用位：
  - 1：清除使用位并清除计数器（在最后一次扫过时被使用）
  - 0：增加计数器；如果计数=N，则替换页面
- 这意味着时钟指针需要扫过N次而页面没有被使用后才会替换该页面

NRU 最不经常使用 - 增强二次机会算法

通过两个状态位—引用位和修改位来近似LRU（即Not Recently Used，NRU算法）：为那些被**修改**的页面赋予更高的优先级以减少I/O负担（增强的第二次机会）。

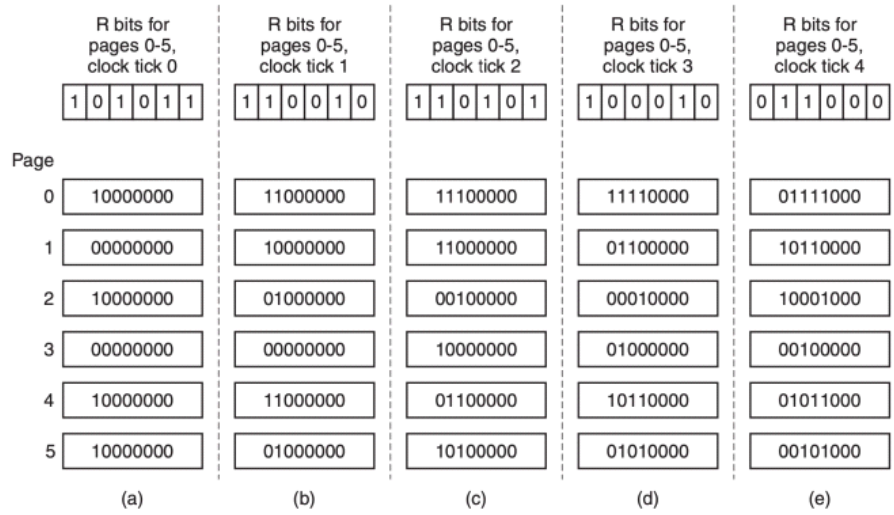
- (0, 0)：既不是最近使用的也没有被修改过（最适合替换）
- (0, 1)：不是最近使用的但被修改过（不太理想，需要在替换前写出）
- (1, 0)：最近被使用但是干净的（但可能会很快再次被使用）
- (1, 1)：最近被使用且被修改过（可能会很快再次被使用，并且需要在替换前写出）

从最低编号的非空类中随机移除一个页面（可能需要多次在循环队列中搜索）

Aging 老化算法

通过额外的引用位来近似LRU（即Aging算法）：保持一个软件计数器来追踪每个页面被引用的次数，并替换计数最小的页面。

- 每个页面关联一个软件计数器
- 在每个时钟中断时
  - 对每个页面的计数器进行右移1位操作（类似于时间/2）
  - 并将R位添加到最左边（类似于新引入的刷新了时间）
- 较大的值表示最近使用的页面
  - 选择01110111页面进行替换，而不是11000100页面



抖动 & 局部性原理 （字面的意思）

抖动（也叫颠簸）：一个进程花费所有时间在页面间进行交换（大多数引用导致缺页异常）

工作集模型算法 —— 局部性的应用

所有 或者 一无所有！！！

工作集（Working Set）：

- 其在时间段(t-x,t)内使用的内存页集合也被视为其在未来(下一个x时间内)会访问的页集
- 如果整个工作集都在内存中，那么进程将运行而不会引起太多缺页异常，直到它进入另一个执行阶段。
- 如果可用内存太小，无法容纳整个工作集，则会发生抖动。

all-or-nothing模型

- 进程工作集要不都在内存中，否则全都换出
- 需要跟踪每个进程的工作集，并确保在运行之前将其加载到内存中。
- 大大降低缺页异常率

跟踪工作集 —— 定时扫描

工作集时钟中断固定间隔发生，处理函数扫描内存页

- 访问位为1则说明在此次tick中被访问， 记录上次使用时间为当前时间
- 访问位为0(此次tick中未访问)
  - Age = 当前时间 – 上次使用时间
  - 若Age大于设置的x，则不在工作集
- 将所有访问位清0
  - 注意访问位(access bit)需要硬件支持

替换策略：本地和全局策略

本地页面置换：每个进程从其分配的帧集中选择受害者

- 每个进程的帧数固定分配
- 每个进程的性能更加一致
- 但可能导致内存利用不足

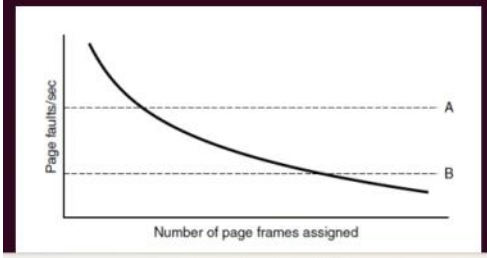
全局页面置换：从分配给任何进程的帧中选择受害者

- 每个进程的帧数可变
- 吞吐量更大，因此更为普遍

- 采用全局页面置换时，操作系统必须不断决定为每个进程分配的页面帧数：
  - 平均分配：为每个进程分配相等的份额。
  - 比例分配：根据进程大小进行分配（需要为每个进程分配一定数量的帧）。
  - 按照优先级分配（优先级高的）
- 这些静态的帧数分配无法解决不断变化的动态需求

缺页异常频率（PFF）算法用于帧分配

- 缺页异常率 = 每秒平均缺页数
- 分配帧以使进程的PFF相等
- 仍然需要临时将某些进程完全交换出去（某些进程需要更多内存，但没有进程需要更少内存）



如果实际速率太低，则进程丢失帧
如果实际速率太高，则进程获得帧

TLB 快表（OS介绍的是全相联）

一个典型的TLB缓存的项包括

- 页号及其对应的帧号
- 有效位：条目是否有有效的转换(注：和Page Entry的有效位意义不同！)
- 保护位：页面的访问方式
- 脏（修改）位：页面是否已被修改

Valid	Virtual page	Modified	Protection	Physical frame
1	140	1	RW	31
1	20	0	R X	32
1	130	1	R	38
1	860	1	RW	101

在TLB缺失时，值被加载到TLB中，以便下次更快地访问。

但如果TLB已满，应该替换谁呢?

- 先来先出?
- 最近最被少引用的(Least-Recently-Used，LRU)?
- 随机?

谁来处理TLB miss?

TLB 一致性

上下文切换时，旧进程的虚拟到物理地址的转换应该不再有效，否则就会出现同样的虚拟地址映射到不同物理地址的问题，解决方案有两种：

- 清空TLB：简单地将所有有效位设置为0
- 带标记的TLB：在每个TLB条目中添加一个地址空间标识符（ASID）字段，该字段唯一地标识每个进程，为该进程提供地址空间保护

VPN	PFN	valid	prot	ASID
10	100	1	rwx	1
—	—	0	—	—
10	170	1	rwx	2
—	—	0	—	—

25. 假定一个计算机系统中有一个 TLB 和一个 L1 数据 cache。该系统按字节编址，虚拟地址 16 位，物理地址 12 位，页大小为 128B；TLB 采用 4 路组相联方式，共有 16 个页表项；L1 数据 cache 采用直接映射方式，块大小为 4B，共 16 行。在系统运行到某一时刻时，TLB、页表和 L1 数据 cache 中的部分内容如下：

组号	标记	页框号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号	有效位
0	03	—	0	09	0D	1	00	—	0	07	02	1
1	13	2D	1	02	—	0	04	—	0	0A	—	0
2	02	—	0	08	—	0	06	—	0	03	—	0
3	07	—	0	63	0D	1	0A	34	1	72	—	0

（a）TLB(4 路组相联)：4 组、16 个页表项

虚页号	页框号	有效位
00	08	1
01	03	1
02	14	1
03	02	1
04	—	0
05	16	1
06	—	0
07	07	1
08	13	1
09	17	1
0A	09	1
0B	—	0
0C	19	1
0D	—	0
0E	11	1
0F	0D	1

(b) 部分页表(开始 16 项)

行索引	标记	有效位	字节 3	字节 2	字节 1	字节 0
0	19	1	12	56	C9	AC
1	—	0	—	—	—	—
2	1B	1	03	45	12	CD
3	—	0	—	—	—	—
4	32	1	23	34	C2	2A
5	0D	1	46	67	23	3D
6	—	0	—	—	—	—
7	16	1	12	54	65	DC
8	24	1	23	62	12	3A
9	—	0	—	—	—	—
A	2D	1	43	62	23	C3
B	—	0	—	—	—	—
C	12	1	76	83	21	35
D	16	1	A3	F4	23	11
E	33	1	2D	4A	45	55
F	—	0	—	—	—	—

(c) L1 数据 cache：直接映射，共 16 行，块大小为 4B

请问(假定图中数据都为十六进制形式)：

(1) 虚拟地址中哪几位表示虚拟页号？哪几位表示页内偏移量？虚拟页号中哪几位表示 TLB 标记？哪几位表示 TLB 索引？

(2) 物理地址中哪几位表示物理页号？哪几位表示页内偏移量？

(3) 主存物理地址如何划分成标记字段、行索引字段和块内地址字段？

(4) CPU 从地址 067AH 中取出的值为多少？说明 CPU 读取地址 067AH 中内容的过程。

(1) 页大小为128B故而低7位表示页内偏移量；

故而，高9位表示虚拟页号；

虚页号中，低2位为TLB组索引(因为16项4路共有4组)，高7位为TLB标记

(2) 低7位表示页内偏移量，故而高5位表示物理页号

(3) 因为物理地址为12位，Cache直接映射，

故而从高到低 12 = 6(标记) + 4(Cache行索引) + 2(块内地址)

(4) 0x067A = 0000 0110 0111 1010

得到 虚拟页号 = 0x03 TLB标记 + 00 组索引，查询快表失败；

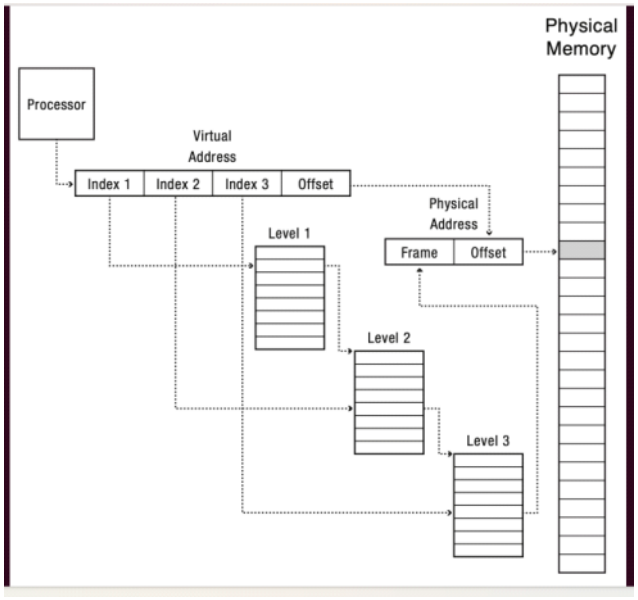
进入主存中页表查询 0x00C 查询成功，找到物理页号0x19 = 1 1001

得到物理地址 1100 1111 1010

得到 0x33标记；0xE行索引；10块内地址(字节2)

查询cache得 0x4A = 0100 1010

多级页表



虚拟大小 VASize
单页大小 PageSize
页框项　　InfoSize
需要 OriginSize = InfoSize * (VASize / PageSize) 大小的空间储存信息。
OriginSize / InfoSize = 基础页表项
PageSize / InfoSize = 一个高级页能管几个低级页
OriginSize / PageSize = 需要几个高级页
=> 取 log 得 页目录索引

不如找个题目（看试卷）

持久化

驱动程序、DMA、软链接、硬链接、inode、多级索引、日志、磁盘原理

驱动程序

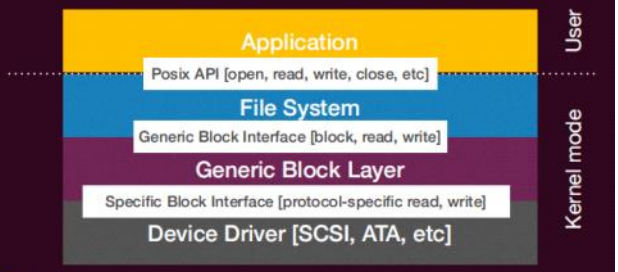
硬件抽象

操作系统创建了一个分层的存储视图

- 即使在内核内部抽象也是不断使用的技术！

分层方法允许更低层部分容易更改

- 例如，文件系统的实现独立于磁盘类型



管理 I/O 设备的代码在内核设备**驱动程序(device drivers)**中

- 70%的 Linux 代码是设备驱动程序，因为有太多不同的设备，每个设备都不同！（1530万行源代码！）
- 设备驱动程序是内核错误的最常见来源
  - 这是由于给定的驱动程序可能只被少数系统使用，所以它不会被高度使用或仔细检查。

设备驱动程序通常分为两部分：

- 上半部分：在系统调用的调用路径中访问
  - 实现一组标准的、跨设备的调用，如 open()、close()、read()、write()、ioctl()
  - 这是内核与设备驱动程序的接口
  - 上半部分将启动设备的 I/O 操作，可能会让线程休眠直到完成 (同步和异步I/O!)
- 下半部分：作为中断例程运行
  - 获取输入或传输下一块输出
  - 如果 I/O 现在完成，可能会唤醒休眠的线程

DMA —— 专门负责memcpy()的CPU

在发出与设备通信的指令后，操作系统需要知道以下情况：

- I/O 设备已完成一个操作
- I/O 操作遇到了错误

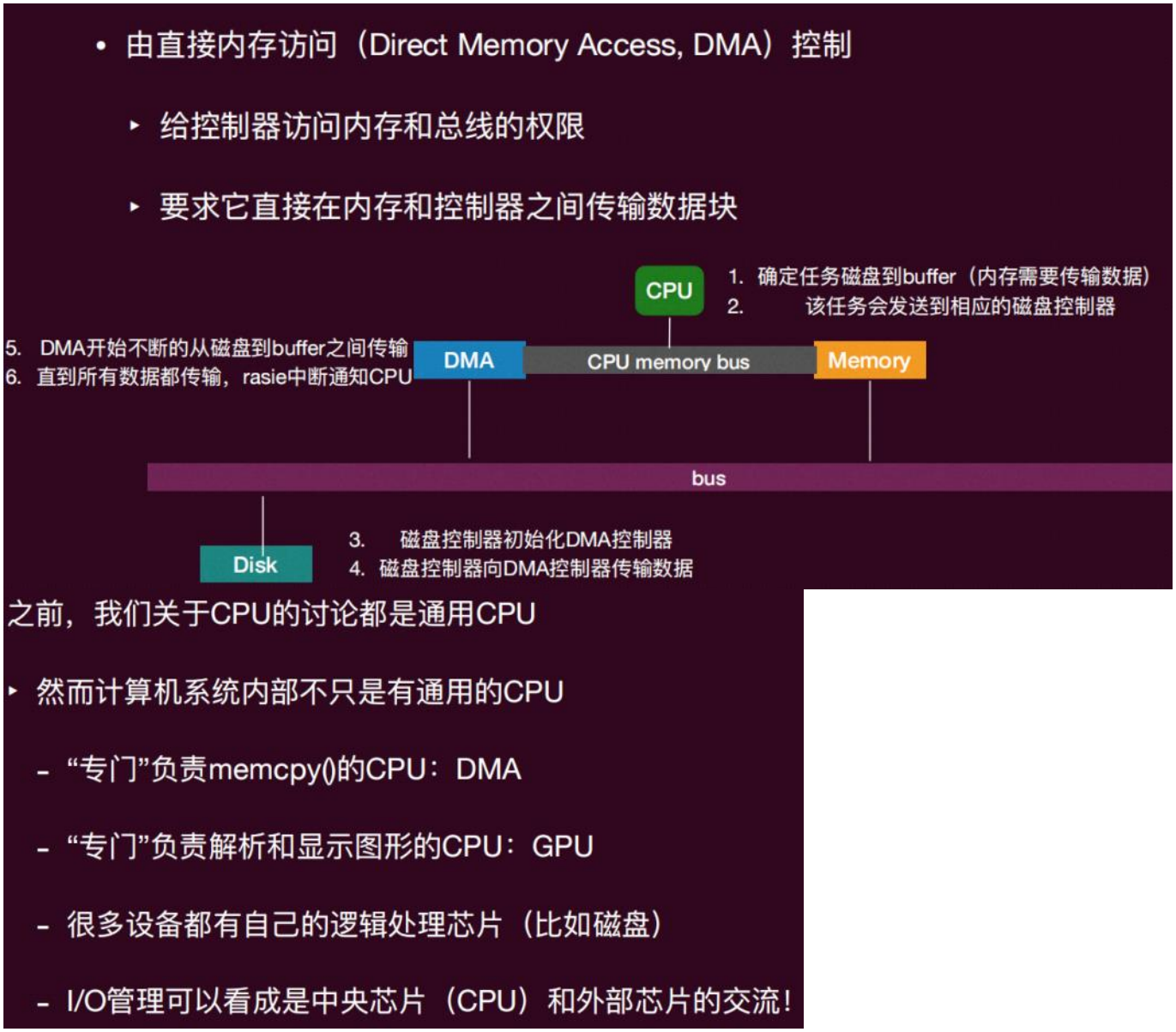
有两种获知状态的方法：

- 轮询(polling)：操作系统定期检查设备特定的状态寄存器
  - 开销低（没有切换），但如果是低速的设备上会浪费CPU周期
- I/O 中断：设备在需要服务时生成中断
  - 不会浪费CPU周期，但是开销高（伴随切换）
- 一般来说两种混合

I / O 数据传输控制：

由CPU来控制：

- 即Programmed I/O
  - 直接通过处理器的in/out或load/store传输每个字节
  - 优点：硬件简单，易于编程
  - 消耗与数据大小成比例的处理器周期（因为每个数据的in/out都要一个指令）
  - 此外，CPU会一直接收到interrupt，速度变慢



DMA（Direct Memory Access，直接内存访问）是一种允许某些硬件子系统在不经过CPU的直接干预下，将自己直接与系统内存进行数据交换的技术。DMA主要用于高速数据传输，以减轻CPU的负担，提高系统的整体性能。

以下是DMA的一些关键特性：

1. **高速数据传输**：DMA允许数据在内存和硬件设备（如硬盘、网络接口卡等）之间高速传输。
2. **CPU释放**：在DMA传输过程中，CPU可以继续执行其他任务，因为数据传输由DMA控制器管理。
3. **减少CPU负载**：由于DMA可以独立于CPU进行数据传输，因此减少了CPU处理数据传输的负载。
4. **内存映射I/O**：DMA通常通过内存映射I/O（Memory-Mapped I/O）进行操作，这意味着某些I/O设备的地址空间被映射到内存地址空间中。
5. **DMA控制器**：每个支持DMA的设备都有一个DMA控制器，它负责管理数据传输。
6. **中断**：在数据传输完成后，DMA控制器通常会通过中断机制通知CPU，以便CPU可以执行后续操作。
7. **数据缓冲**：DMA传输通常使用缓冲区（Buffer）来暂存数据，然后一次性传输到内存，这样可以减少访问内存的次数。
8. **优先级和仲裁**：在多个设备都需要使用DMA时，系统需要一个仲裁机制来决定哪个DMA请求先被服务。
9. **编程和配置**：DMA传输需要编程和配置，包括设置源地址、目标地址、传输大小和传输模式等。
10. **硬件和软件支持**：DMA需要硬件和软件的支持。硬件需要有DMA控制器，软件需要能够配置和使用DMA。

DMA是现代计算机系统中不可或缺的一部分，特别是在处理大量数据传输时，如视频播放、音频处理、网络通信等场景。通过使用DMA，可以显著提高这些操作的性能和效率。

软链接 & 硬链接

硬链接（Hard Link）

1. **定义**：硬链接直接指向文件的inode（即文件的物理存储位置），而不是指向文件名。
2. **特点**：
    - 不能跨文件系统创建：硬链接必须与原文件在同一文件系统中。
    - 删除文件名不会影响硬链接：即使原文件被删除，硬链接仍然有效，因为它们指向同一个inode。
    - 不允许链接到目录，以免循环。

软链接（Symbolic Link）

1. **定义**：软链接是一种特殊类型的文件，它包含了指向另一个文件或目录的路径，寻inode时，需要得到路径后，根据真实路径寻inode。
2. **特点**：
    - 可以跨文件系统：软链接可以指向不同文件系统中的文件或目录。
    - 删除原文件会导致软链接失效：如果原文件被删除或者改名，软链接将指向一个不存在的文件。
    - 软链接可以指向不存在的文件：即使目标文件不存在，软链接也可以被创建，但会产生引用悬空。
    - 软链接可以指向目录：可以为目录创建软链接，但删除软链接不会影响它指向的目录。

inode

Inode 简介：

文件元数据：

除了文件的名称和数据外，操作系统还会保留文件的额外信息：

- 位置：文件在设备上的位置指针
- 大小：文件的大小（当前大小或最大大小）
- 时间：文件的创建时间、最近访问时间和最近修改时间
- 所有者：文件的当前所有者
- 保护信息

文件控制块

文件系统应该将文件元数据保存在一个结构中（文件控制块）：

- 存储在磁盘上，并且缓存在内存中以加快访问速度。

在UNIX中，这个文件控制块就是 inode（index node）

文件描述符

文件描述符（句柄）：操作系统分配给文件的一个唯一数字（每个进程私有），用于引用该文件。

- 具有执行特定操作的能力
- 避免在每次访问时解析文件名（在目录中搜索文件名）和检查权限
- 一个文件可以以不同方式多次打开

文件偏移

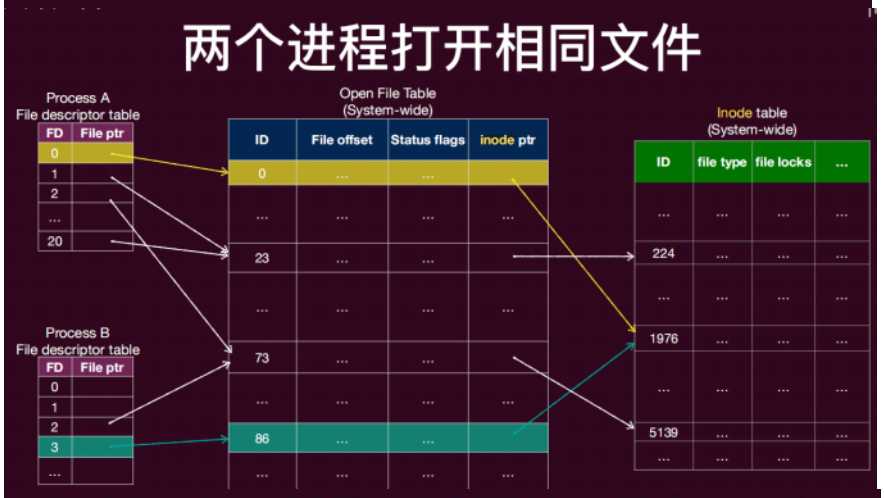
对于进程打开的每个文件，操作系统都会跟踪一个文件偏移量，该偏移量决定下一次读取或写入将从何处开始。

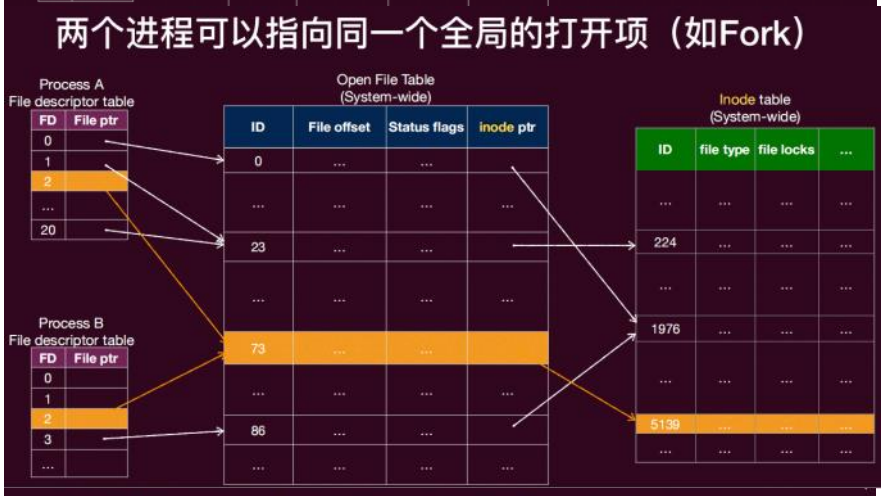
- 隐式更新：当进行N字节的读取或写入时，N会被添加到当前偏移量。
- 显式更新：使用lseek()函数。

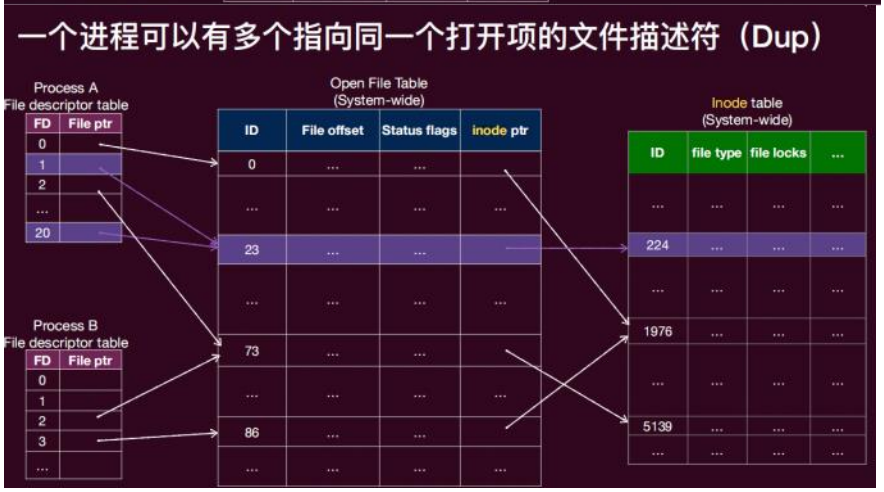
三张表

在Unix中，有三种内核数据结构用于描述文件描述符和打开文件之间的关系：

- 文件描述符表（每个进程）：为进程打开的每个文件描述符创建一个条目
  - 指向打开文件表的指针
- 打开文件表（系统范围）：为每个打开的文件创建一个条目
  - 文件偏移量、访问模式、与文件打开相关的标志
  - 指向inode的指针
- inode表（系统范围）：为每个inode点创建一个条目
  - 文件属性（类型、大小、权限、时间戳等）







目录：

目录存储了文件名与低级别结构（文件控制块）之间的映射

- 在 Unix 中，每个目录条目只是一个〈文件名，inode 号〉对
- 目录被存储为一个文件
- 要查找一个文件，需要找到包含该映射的目录
- 根目录是特别的：需要为根目录分配一个固定的 inode 号

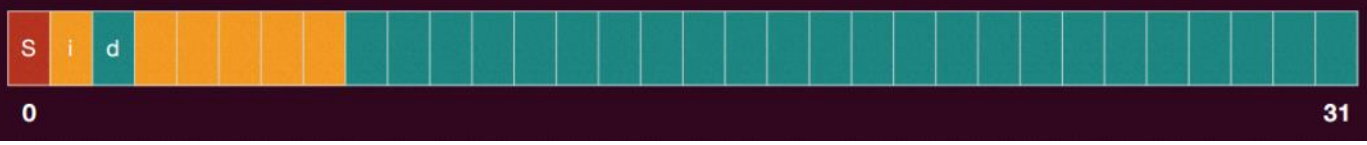
.	1952
..	6253
tmp	224
test.txt	1976
main.c	4594

寻找 /usr/ast/mbox 的步骤



文件系统实现（不重要？）

- 文件系统需要为需要存储的数据维护一个其在磁盘上的数据结构
  - 超级块（Superblock）：文件系统本身的信息
    - 文件系统的类型、inodes数量、数据块数量、inode表开始的块地址…
    - 当挂载一个文件系统时会读取超级块的信息



多级索引 —— 文件组织

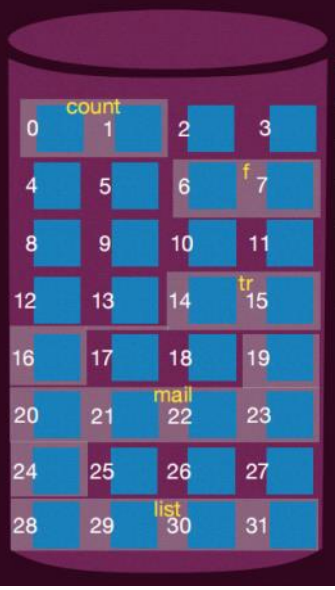
文件需要磁盘给定相应的数据块进行存储，此外还需要有一些数据结构来反映该文件用的数据块在哪

很多种组织方式

- 连续存储（Contiguous）
- 链表（Linked List）
- 文件分配表（File Allocation Table）
- 索引式配置（Indexed Allocation）

连续分配：

- 在连续分配下：
  - 只有文件的第一个数据块的地址和需要的数据块总数需要记录
  - 线性的访问是高效的
  - 随机访问的数据地址也是容易计算的



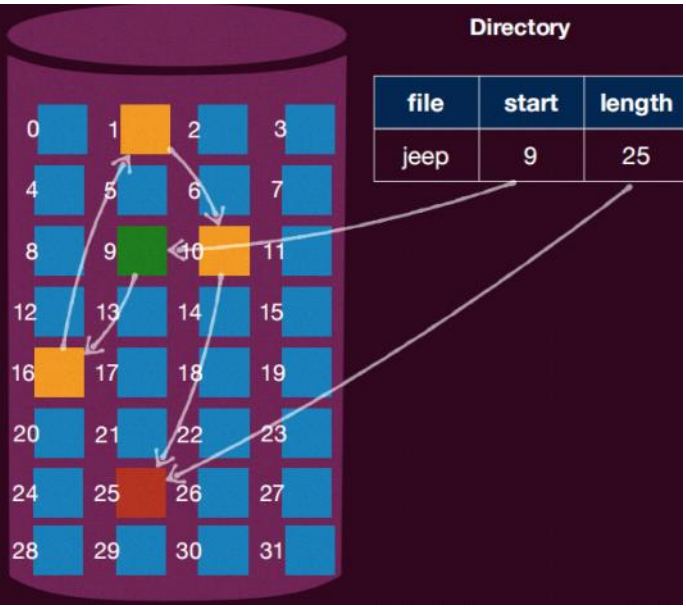
Directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

链表分配 不是length 应该是end

每个文件是一个数据块的链表

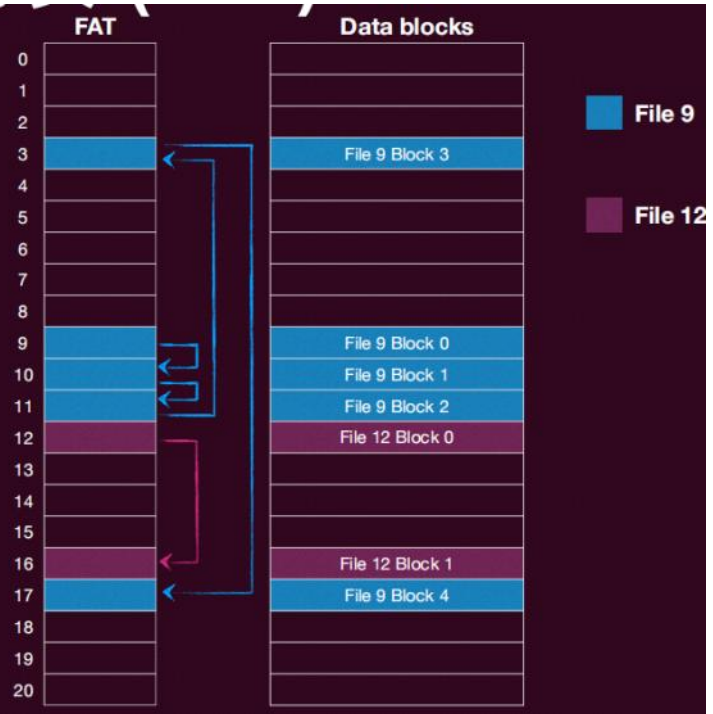
- 磁盘上的每个数据块都可以被使用
  - 没有外部碎片
- 仅仅需要存储第一个块的地址
- 然而，
  - 随机读取会产生很多磁盘I/O
  - 每个数据块都需要为指针留取空间
  - 可靠性问题：如果某个指针损坏...



文件分配表： 链表的改进

文件分配表（FAT）：一个基于链表分配的变种，其将所有的指针放到同一个表格中

- 对于每个磁盘数据块都有一个FAT的项
- 每个FAT项包含一个指向下一个FAT项的指针，或者文件终止符号
- 对于随机访问的操作，只需要访问该FAT表即可，该表可以直接存储进主存从而减少需要I/O的次数



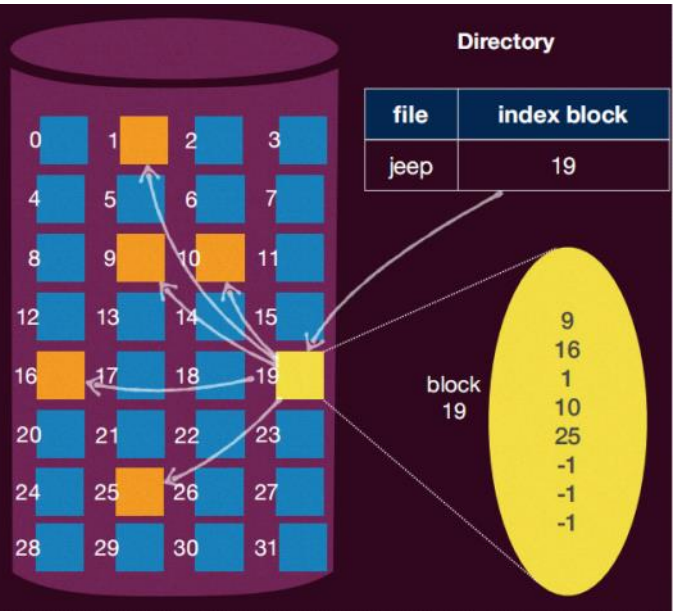
- FAT文件系统布局
  - FAT表还可以用来进行空闲空间追踪
  - 可以使用两个FAT表来增加可靠性
- 但FAT需要存储在主存中，这会带来巨大的开销
  - 对于容量为一个1TB (2^{40}) 的和块大小为4KB (2^{12}) 的磁盘来说，需要2^{28}的表项;
    - 如果每个表项需要4个字节，那么一个FAT表格就需要1GB



索引式

每个文件有一个对应的指针数组块（索引数据块），其中的每个指针指向该文件的一个数据块

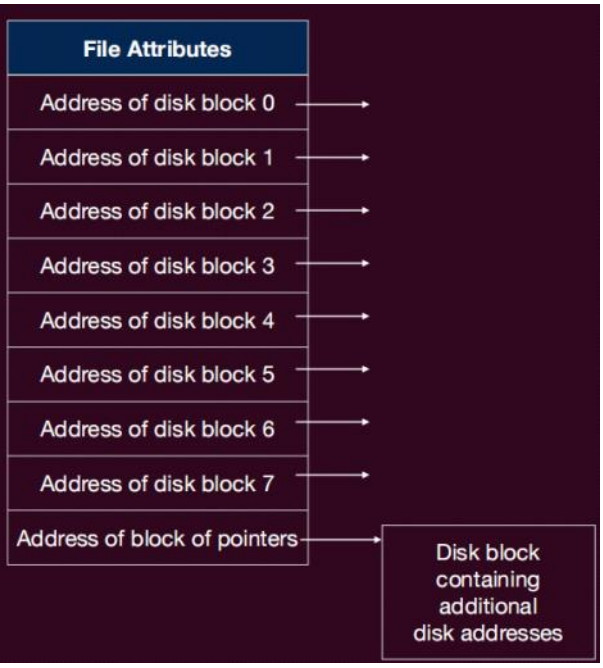
- 第i个指针指向文件的第i个磁盘数据块
- 当文件打开时，该索引块才会被加载进内存
- 因此内存中的开销只和打开文件数相关，而不是整个磁盘
- 但有一个问题：每个文件所需要的指针个数不同（占据的数据块的个数不同），从而导致索引块大小难以确定



多级索引!

使用间接指针：可以将索引块中的指针指向一一个由指针组成的数据块，其中每个指针再指向数据块

- 一个索引块中可以赋予固定数量的直接指针（指向数据块）和固定数量的间接指针
- 此时索引的结构变成了一个树（非平衡）
- 对于小文件和大文件都能很好支持
- 一般有2级和3级的非间接指针



假定数据块为4KB，一个表项为4字节

一个索引块包含15个指针，Index structure contains a set of 15 pointers

- 开始的12指针指向数据块 (12 * 4 KB = 48KB)
- 最后3个指向间接数据块
  - #13: 单级间接指针 (2^{10} * 4 KB = 4MB)
  - #14: 双级间接指针 (2^{10} * 2^{10} * 4 KB = 4GB)
  - #15: 三级间接指针r (2^{10} * 2^{10} * 2^{10} * 4 KB = 4TB)

$2^{10} = 4KB / 4$

日志 PPT 14 P115 - P124

基本思路：预写日志（Write-ahead logging）或日志（Journaling），该思路借鉴自数据库系统

- 在覆盖结构之前，先写一个小日志（存储在磁盘上），描述将要做的事情
- 如果在更新过程中发生故障，我们可以在重启时读取日志并重试
  - 在写入意图之前崩溃：没有操作
  - 在写入意图之后崩溃：重做该操作
- 在更新期间增加的一些工作量，可以大大减少恢复期间所需的工作量
  - 无需扫描整个磁盘，只需要查看奔溃前的日志中的记录即可
- 被许多文件系统使用，包括 Linux 的 ext3、ext4 和 Windows 的 NTFS

文件的更新被作为事务（transaction）保存在日志中（日志写入）

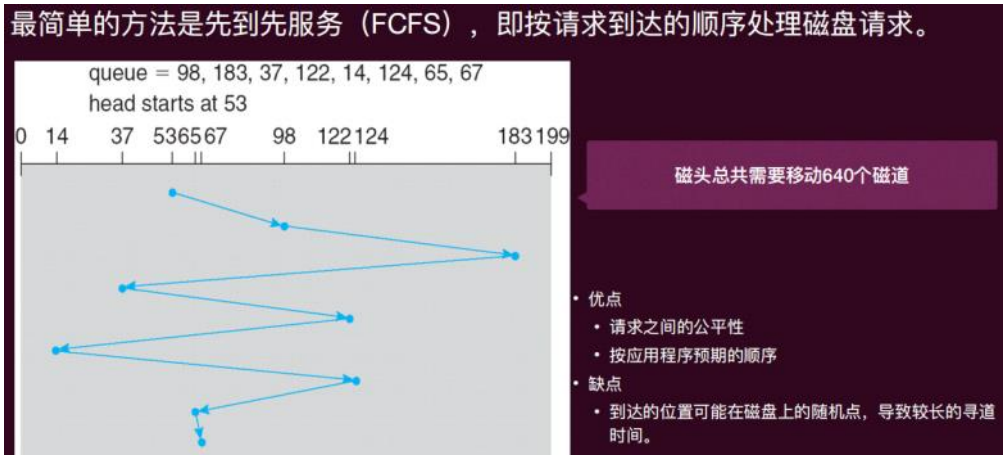
- 写入 TxB（transaction begin，事务开始），其包括事务标识符和其他相关信息（例如，块I[v2]、B[v2]和Db的地址）
- 将inode（I[v2]）、位图（B[v2]）和数据块（Db）的具体更新放入日志中（待处理更新）
- 写入 TxE（transaction end事务结束）



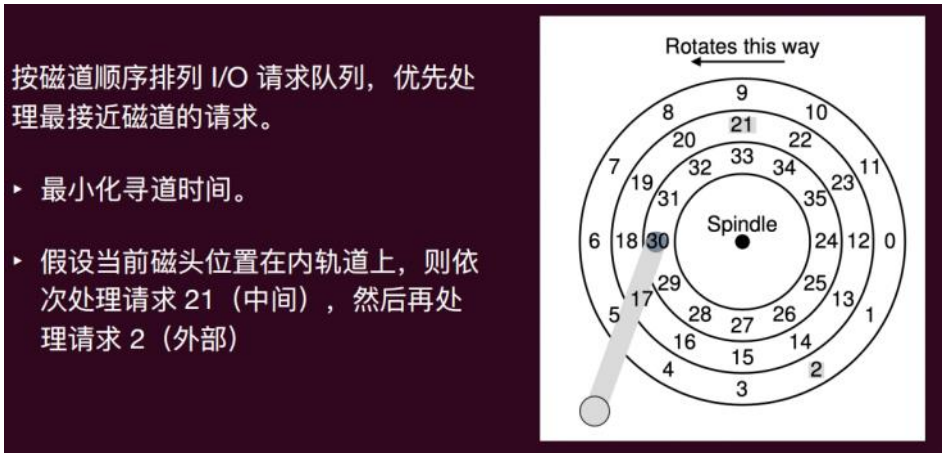
磁盘原理

磁盘调度

FCFS

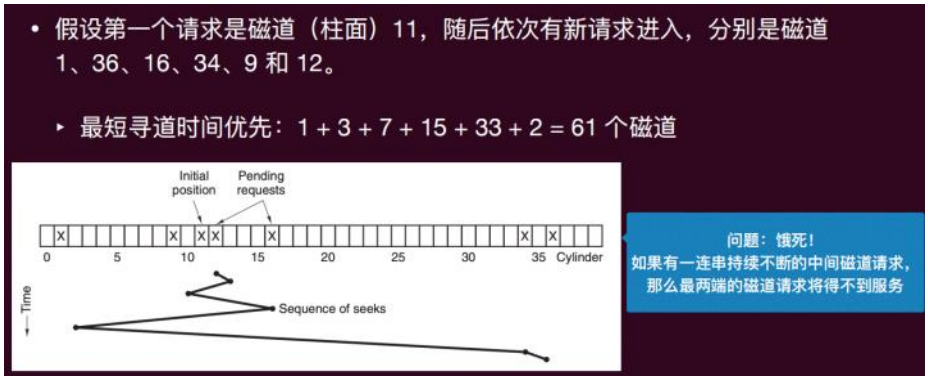


SSTF



这个配图不太好，右上图的数字代表的是请求序号；

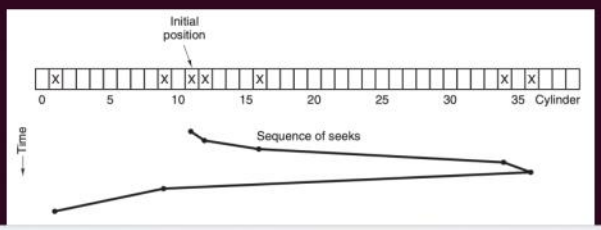
下面图中的数字是磁道序号，一个磁道是一个同心圆环



SCAN 电梯算法

简单地在磁盘上来回移动，按顺序跨磁道处理请求（像电梯一样运行）。

- 从磁盘的一端（从外轨到内轨或从内轨到外轨）的单次通过称为一次扫描。
- 磁臂移动距离为：1 + 4 + 18 + 2 + 27 + 8 = 60 个磁道。



F-SCAN

在执行扫描时暂时冻结要处理的队列（避免远距离请求的饥饿）

C-SCAN

只从外轨到内轨扫描，然后重置到外轨重新开始，

更平均的等待时间，即对内轨和外轨更公平。

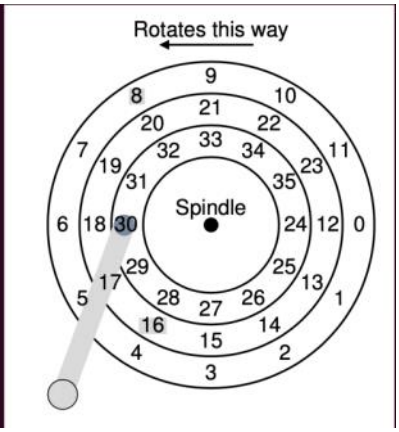
SPTF 最短定位时间

同时考虑寻道时间和旋转时间。

假设当前磁头位置在第 30 扇区（内轨），那么是处理请求 16（中间）还是请求 8（外轨）？

- 如果寻道时间远高于旋转延迟，那么最短寻道优先（SSF）是好的选择。
- 但如果寻道速度比旋转快得多，那么应先处理请求 8。

在操作系统中实现这一点很困难，因为操作系统通常不清磁头当前的具体位置



固态硬盘 SSD

充电/放电实现 1-bit 信息的存储

间接层

- 在固态硬盘（SSD）中维护一个闪存转换层（FTL）。
- 将虚拟块编号（操作系统使用的）映射到物理页面编号（闪存存储控制器使用的）。
  - 现在可以自由重新定位数据，而无需操作系统知道。

写时复制

- 当操作系统更新其数据时，不要覆盖页面（这样做很慢，需要先擦除页面，而擦除的效率很低，比写入低）
- 相反，将新版本写入一个空闲页面。
- 更新 FTL 映射以指向新位置。

固态硬盘（SSD）控制器可以分配映射以在页面之间均衡工作负载

- 均衡磨损（Wear Leveling）

对旧版本页面应该怎么处理?

- 在后台进行垃圾收集
- 擦除带有旧页面的块，将其添加到空闲列表

Copy-on-write 意味着旧的数据还在!

- logic block 被覆盖，physical block 依然存储了数据
  - 有研究发现实际的智能手机的一些被删除的数据之后仍然“存在”超过六个月

起度格式化同样存在相同的问题